Differential Expression Method Comparison on Data from Optical Pooled Screens in Human Cells by Feldman et al. (2019)

Cytokine Induced Temporal Transcriptomic Alterations in 1 hour vs. 4 hours in MED12 KO and MED24 KO cell lines

Nuri Maciel Morales Cornejo - 2182650 Lisa De Rossi - 2203593

2026-05-25

Contents

Biological problem **2**

Objective **2**

Experimental design **3**

Wet-lab protocol . . . 3

Miami compliant . . . 3

Data Processing and Analysis **4**

Data exploration . . . 4

Loading data . . . 4

Sample's Info parsing . . . 4

Diagnostic plots . . . 5

Confounding factors analysis . . . 8

Exploratory PCA . . . 11

GC content gene length . . . 14

Matrices filtering . . . 14

Normalization . . . 17

Initialization of data structure SeqExpressionSet . . . 18

Between lane Normalization . . . 18

Within lane Normalization . . . 19

Global Expression Distribution Diagnostics Across Six Normalization States . . . 19

Relative Log Expression (RLE) . . . 22

Effect of normalization on library size . . . 25

GC Content Bias Profiling . . . 26

Correlation heatmap (TMM) . . . 29

Differential Expression Analysis **30**
EdgeR . . . 30
DESeq2 . . . 33
NOISeqbio . . . 33
Consensus DEGs . . . 34
Evaluation of Consensus Genes . . . 35
Volcano plots . . . 37
Summary table . . . 40
Inter-Algorithmic Performance Assessment . . . 41

Clustering **42**
Clustering Validation . . . 48
Heatmaps . . . 48

Functional Over-Representation (ORA) **52**
Over-representation dotplots . . . 56
GSEA ridgeline and Pathview . . . 58

Implementation and comparison of different Bioconductor packages **63**
EdgeR . . . 63
DESeq2 . . . 63
NOiseq . . . 64

Biological interpretation **65**

Biological problem

The main biological question that drives the RNA bulk sequencing in this study, is understanding the translocation kinetics of NF- B by measuring the transcriptional consequences associated with the amount of time the p65 subunit remains in the nucleus in both MED12-KO and MED24-KO HeLa cells. However this methodological step is not the main objective of this study, but a validation pipeline of a novel technological approach proposed: Optical Pooled Screening.

Objective

To perform a comparative temporal transcriptomic profiling between 1 hour vs. 4 hour timepoints of MED12-KO and MED24-KO HeLa cells under IL-1 and TNF- stimulation, utilizing a multi-package diferential gene intersection approach to isolate high-confidence deregulated pathways.

Experimental design

Wet-lab protocol

The methodology of Optical Pooled Screens in Human Cells presented by Feldman D. and collaborators explores a novel platform of massive optical screening that allows linking engineered genetic perturbations to cellular phenotypes in millions of fixed cells. The methodology is based on the delivery of CRISPR libraries using lentiviral vectors and in situ sequencing by synthesis using padlock probes followed by rolling circle amplification. This study focused on the NF- B signaling pathway, which was induced by the administration of IL-1 and TNF- , and evaluated nuclear translocation of p65, a NF- B subunit in more than 3 million cells.

A LentiGuide-Puro vector was engineered to carry a sgRNA and a Pol II-transcribed antibiotic resistance gene containing a 12-nt barcode flanked by constant sequences on its 3' UTR region, the authors named this modified version LentiGuide-BC. A low multiplicity infection transduction (MOI < 0.01) was performed to ensure an individual integration per cell. The barcodes expressed as mRNA were detected using targeted in situ sequencing, the mRNA sequences were converted into cDNA and hybridized with padlock probes, ligated by enzymatic ligation and further amplified by rolling circle amplification. These products of amplification underwent a 12-cycle sequencing by synthesis under low magnification microscopy (10x), enabling high-throughput analysis. Sequences obtained from the imaging process were then compared to a predesigned barcode dictionary of 83,314 barcodes, the 12-nt barcode were synthetized using criteria that balanced GC content, minimized homopolymer repeats, and maintained a minimum edit distance between each of them.

To validate the precision of the system, a CRISPR system was implemented on cell lines containing an inactive hemagglutinin (HA) gene frameshift reporter. In this validation workflow the CRISPR-Cas system was designed to target the inactive sequence of the HA gene and correct the frameshift of the protein, thereby producing functional HA, which was later detected using conjugated antibodies. When the in situ sequencing was performed, 90.6% correspondence between the reporter activating RNA guide and its expected barcode, demonstrating a high fidelity and allowing it to be used in large-scale genomic libraries

For results validation and eliminating recombination artifacts a screen beyond static points was performed, using a CROP-seq vector, which expresses a Pol II-transcribed copy of the sgRNA that can be directly read by in situ sequencing without an auxiliary barcode. Cas9 expression was induced by a 14 day doxycycline treatment and cells were loaded with Hoechst 33342 as a live nuclear stain. Cells were then stimulated with IL-1B or TNF-A, and imaged at 23 minute intervals over 6 hours, capturing 361,587 cells. This temporal approach revealed MED12 and MED24 as negative regulators, both prolonged p65 nuclear retention at time points >2 h.

To confirm MED12 and MED24 as bona fide hits, a high-coverage pooled validation screen targeting 61 candidate genes at 10 sgRNAs per gene was performed, confirming both genes with strong intra-gene sgRNA concordance across cells.

Clonal homozygous knockout lines were then generated using synthetic crRNA/tracrRNA duplexes to target MED12 and MED24 in the HeLa-TetR-Cas9/p65-mNeonGreen cell lines. Both homozygosity knockout and the expected p65 nuclear retention were validated by sequencing and live-cell microscopy For bulk RNA sequencing, clones were stimulated with TNF- or IL-1 for 0, 1, or 4 hours and sequenced on a NextSeq 500 Illumina sequencer using a paired-end approach in 36 cycles. Differential expression analysis using edgeR identified upregulation of chemokine genes and and induction of pro-IL-1 by TNF- exclusively in MED12/MED24-deficient cells, implicating Mediator complex subunits as suppressors of the NF- B transcriptional response.

Miami compliant

MINSEQE (Minimum Information About a Next-generation Sequencing Experiment) guidelines are completely met. The documentation and data can be found on the GEO database on with the registry ID "GSE132704".

Data Processing and Analysis

Data exploration

Loading data

```
library(omicsdata)
library(limma)
library(EDASeq)
library(edgeR)
library(dplyr)
library(plyr)
library(corrplot)
library(ggfortify)


#data in summarized experiment
#data = readRDS(("/opt/omicsdata/datasets/SRP201365.rds"))
#saveRDS(data, "DataSet.rds")

#retrieve from directory
data <- readRDS("DataSet.rds")



RawExprMtx = assay(data)
```

Sample's Info parsing

"sra.sample_attributes" contains experimental data in the format:"key|value;;key|value".
We extract the values into a clean data frame.

```
# Extract RAW sample experimental data
RawColumn = colData(data)$sra.sample_attributes

# Splitting "key|value;;key|value" formatting
parse_attrs = function(x) {
  pairs  = strsplit(x, "\\|")[[1]]
  values = sapply(strsplit(pairs, ";;"), `[`, 2)
  values
}

# Applied to all samples, stored in dataframe
ExperimentalInfo = as.data.frame(t(sapply(RawColumn, parse_attrs)),
                                 stringsAsFactors = FALSE
)

# Name assignment, matching sample names
colnames(ExperimentalInfo) = c("cell_line", "clone", "genotype","source_name",
                               "stimulation", "timepoint")

rownames(ExperimentalInfo) = colnames(RawExprMtx)
```

Diagnostic plots

Before executing downstream Differential Expression and Over Representation Analysis, robust quality control (QC) is performed to assess global data distribution, identify potential batch effects, and detect sample outliers.
By evaluating the raw and log-transformed count matrices through visual methods, such as distribution boxplots, Relative Log Expression (RLE) profiles, we can comprehensively diagnose systematic variances and evaluate the homogeneity of the experimental replicates across the monitored timepoints (1h vs 4h).

```
library(ggplot2)
library(ggfortify)


colores <- c("steelblue3", "indianred3", "seagreen4",
             "tan1", "mediumpurple3", "yellowgreen")

ExperimentalInfo$Groups <- paste0(ExperimentalInfo$genotype, ExperimentalInfo$timepoint)

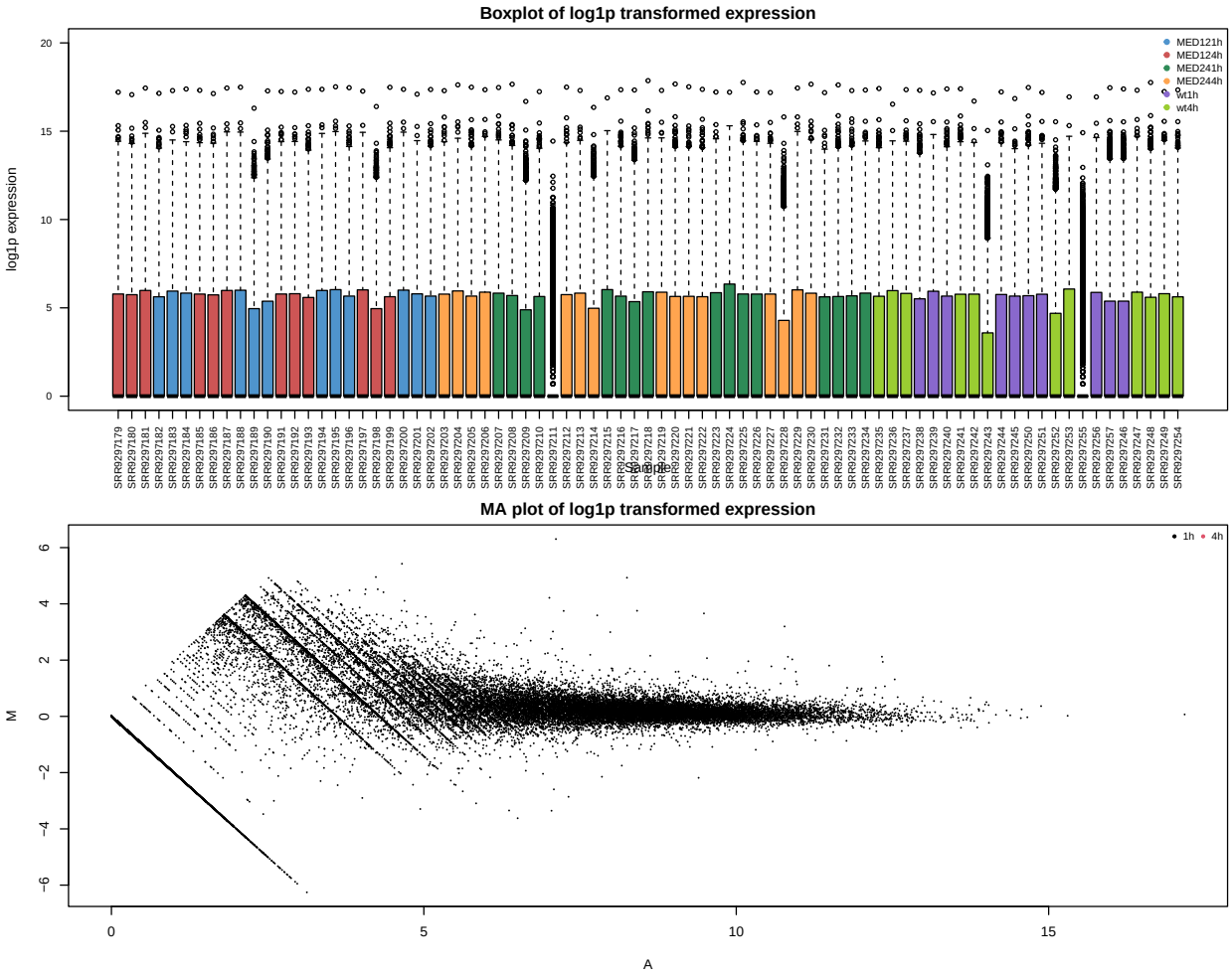
ExperGroups <- sort(unique(ExperimentalInfo$Groups))
colorMap <- setNames(colores, ExperGroups)
mapa_colores <- colorMap[ExperimentalInfo$Groups]
col = mapa_colores


par(mfrow = c(3,1), mar = c(5,4,2,1))

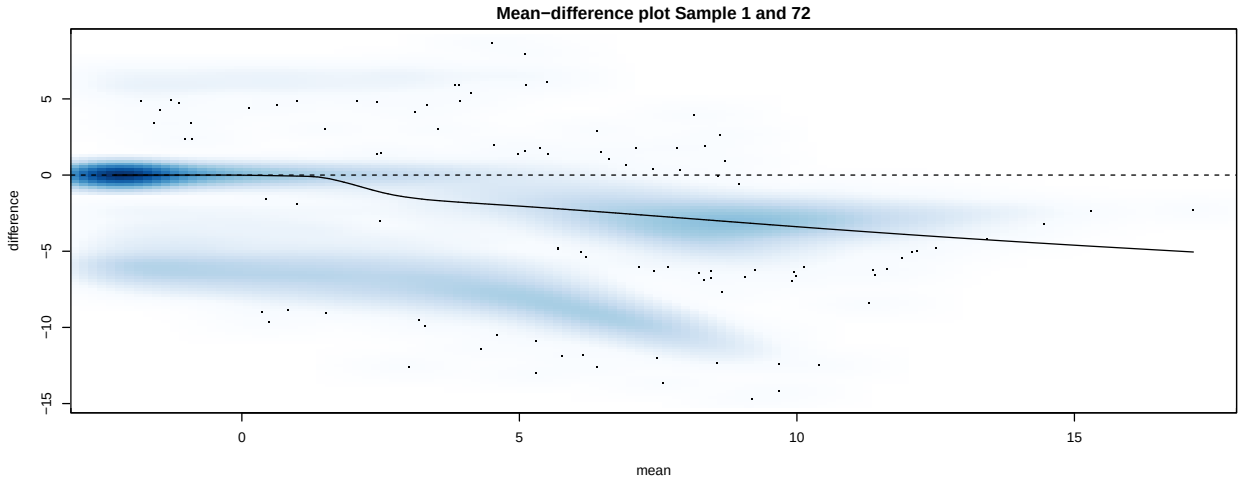
#Boxplot of log transformed counts
#objective: to evaluate and compare the overall distribution of gene expression
#level across all samples. To detect sequencing depth discrepancies
boxplot(log1p(RawExprMtx), col = col, xlab = "Sample", ylab = "log1p expression",
        ylim = c(0, 20), las = 2, cex.axis = 0.7,
        main = "Boxplot of log1p transformed expression")
legend("topright", legend = names(colorMap), col = colorMap, pch = 19, bty = "n", cex = 0.8)

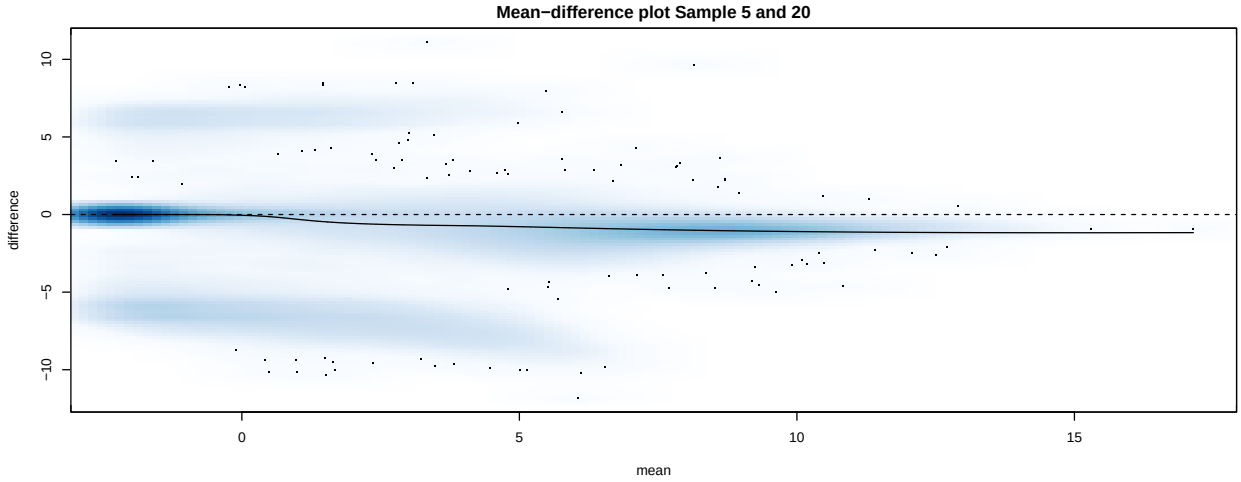

#MA plot
#objective: to detect bias between expression intensity and log-fold change
limma::plotMA(log1p(RawExprMtx), main = "MA plot of log1p transformed expression",
              xlab = "A", ylab = "M")
legend("topright", legend = c("1h","4h"), col = 1:2, pch = 20, cex = 0.7,
       ncol = 2, bty = "n")

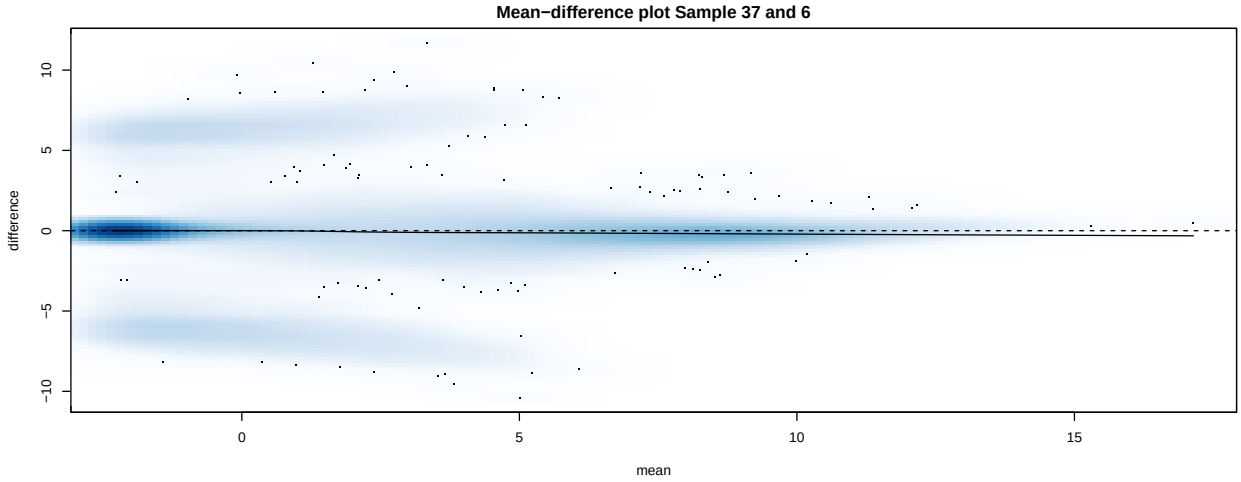
par(mfrow = c(3,1))
```




```
#pairwise mean difference (MD) plot
#to perform targeted comparisons between selected pairs of samples
MDPlot(RawExprMtx, c(1, 72),  main = "Mean-difference plot Sample 1 and 72")
MDPlot(RawExprMtx, c(5, 20),  main = "Mean-difference plot Sample 5 and 20")
MDPlot(RawExprMtx, c(37, 6),  main = "Mean-difference plot Sample 37 and 6")
```







Summary of QC Anomalies

Boxplot Analysis:
The raw library distributions show a consistent median log_{1p} expression level across the majority of the samples. However, three obvious critical outliers exhibit severe distribution collapse: SRR9297211, SRR9297243, and SRR9297255. These three anomalous samples were excluded from the following steps of the pipeline to prevent statistical bias.

MA Plot Analysis: At expression regimes $A > 6$, the data shows symmetry tightly centered around the baseline ($M = 0$), highliting a stable global transcriptome across samples. Conversely, at low expression levels ($A < 5$), diagonal stripes emerge. This striping is a an artifact of log-transformations applied to small discrete integer counts, which are heavily dominated by Poisson noise. These later features lack sufficient statistical power for robust differential modeling, an expression filter (via edgeR's filterByExpr) was applied to delete this before further testing.

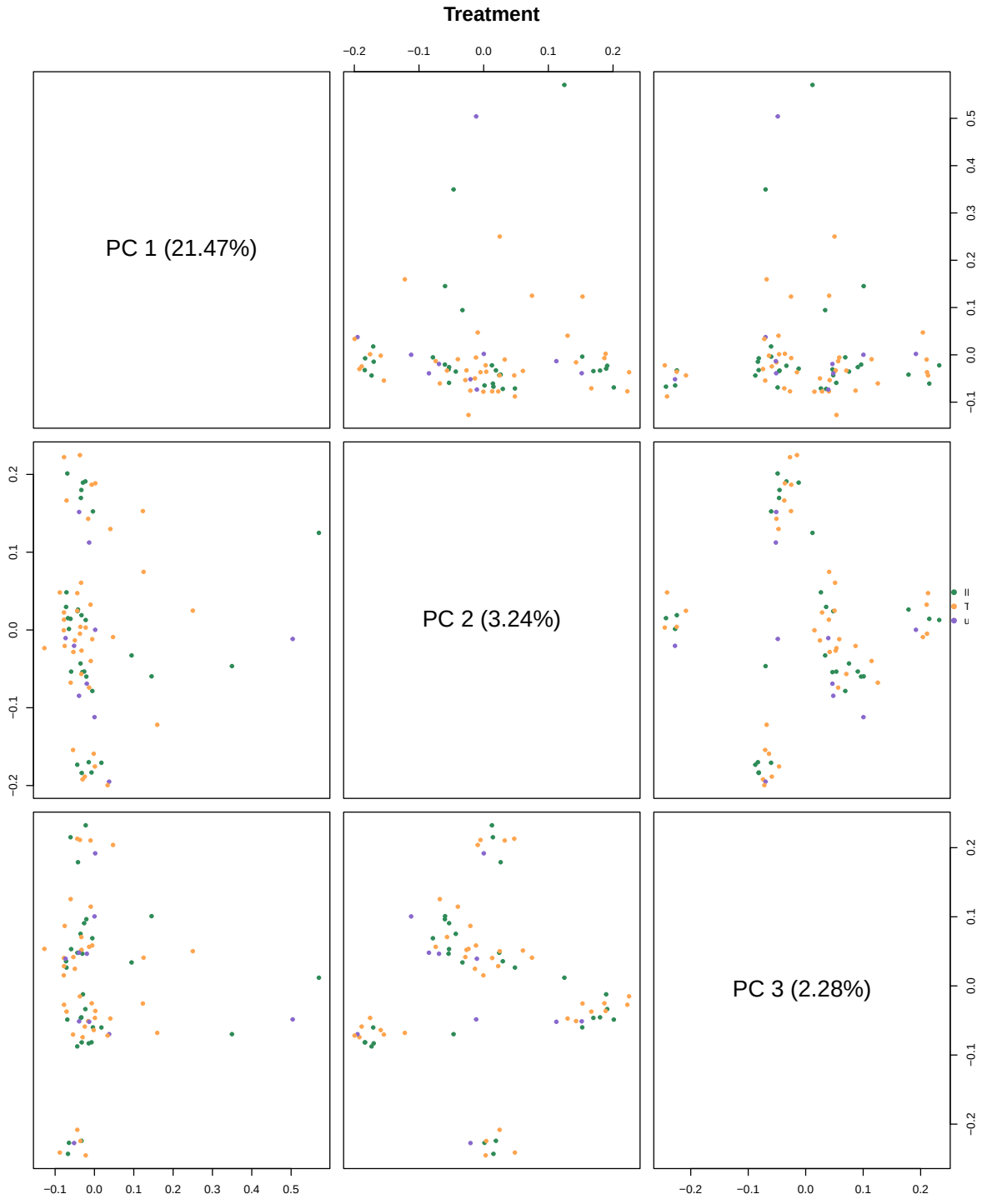
Mean-Difference (MD) Plots Analysis To evaluate pairwise technical variations and the presence of systematic scaling biases between individual libraries, Mean-Difference (MD) plots were generated across three representative sample pairs:
- Sample 1 and 72 (Top panel):
A severe downward trend is visible, with the lowess regression line dropping sharply below the horizontal baseline ($M = 0$) as the mean expression intensity increases. This strong negative skew indicates a severe composition bias or pronounced batch effects between these libraries.
- Sample 5 and 20 (Middle panel):
A mild downward curvature is observed in the high-expression regime ($A > 10$), indicating that although this pair exhibits better alignment than the top panel, minor residual bias persists. This confirms that simple library size scaling (such as CPM) is insufficient to eliminate all non-biological variations, highlighting the need for multi-factor normalization.

- Sample 37 and 6 (Bottom panel):
The ow trend line remains perfectly flat and aligned with the dashed reference line ($M = 0$) across the entire dynamic range of gene expression, presenting a diagnostic result showing very low technical and compositional differences between these two libraries.

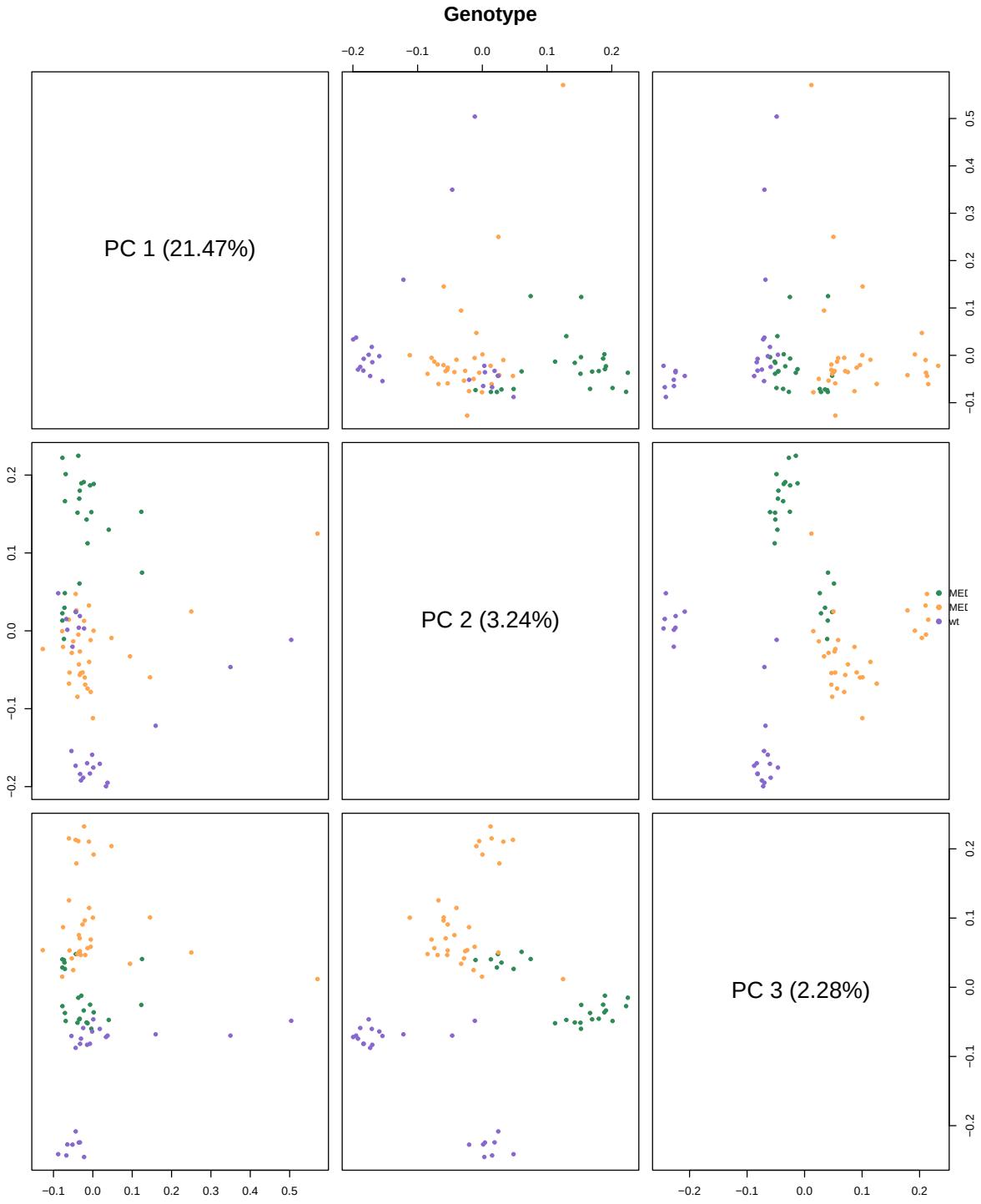
Confounding factors analysis

```
#by tratmeant
COLORS =  c("seagreen4", "tan1", "mediumpurple3")


stimu  = as.factor(substr(ExperimentalInfo$stimulation,1,3))
EDASeq::plotPCA(RawExprMtx, k=3, labels=FALSE, col=COLORS[stimu], pch=20, main = "Treatment")
legend("right", legend = c("IL1", "TNF", "uns"), col = COLORS, pch = 19, bty = "n",cex = .6)
```




```
#by genotype
geno = as.factor(ExperimentalInfo$genotype)
EDASeq::plotPCA(RawExprMtx, k=3, labels=FALSE, col= COLORS[geno], pch=20, main = "Genotype")
legend("right", legend = c( "MED12", "MED24", "wt"),col = COLORS, pch = 19, bty = "n",cex = .6)
```



To identify the dominant drivers of global transcriptional variance and assess potential technical batch effects, exploratory Principal Component Analysis (PCA) was performed across the dataset.

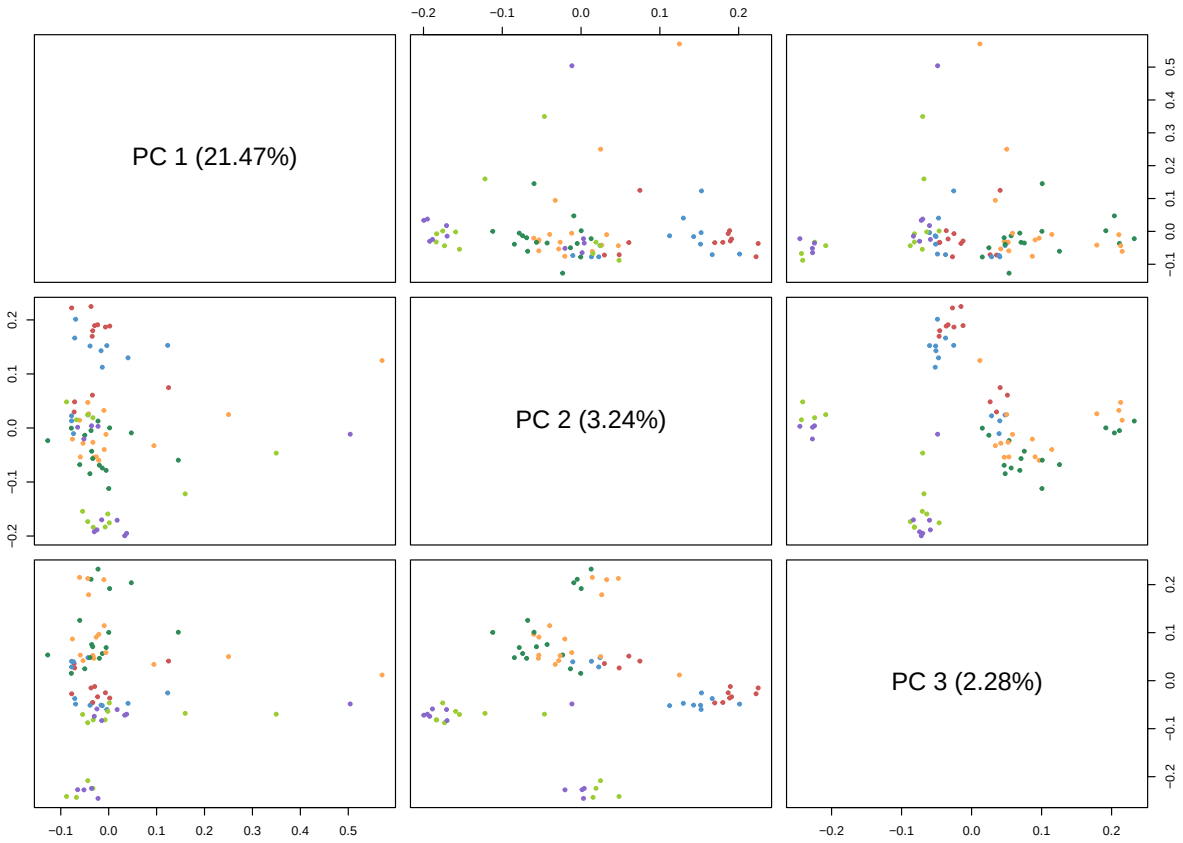
Variance Distribution: Principal Component 1 (PC1) accounts for the vast majority of total transcriptomic variance (21.47%), whereas PC2 (3.24%) and PC3 (2.28%) only capture (3.24%) and (2.28%) respectively, but counter intuitively are the two PC that better split the groups in biological meaningful manner.

Treatment Confounding Factor (First Figure): Visual overlay of treatment conditions (IL-1, TNF-α, and untreated) lack of spatial separation demonstrates that exogenous cytokine treatments do not introduce significant global batch effects.

Genotype Biological Driver (Second Figure): In contrast to the treatment profiles, clear and distinct sample clustering is visible along the PC2 and PC3 coordinates. The wild-type (WT), MED12-KO, and MED24-KO groups split into obvious isolated populations. These diagnostics confirm that transcriptional variance is reliably driven by the primary biological variable of interest: the cell Genotype.

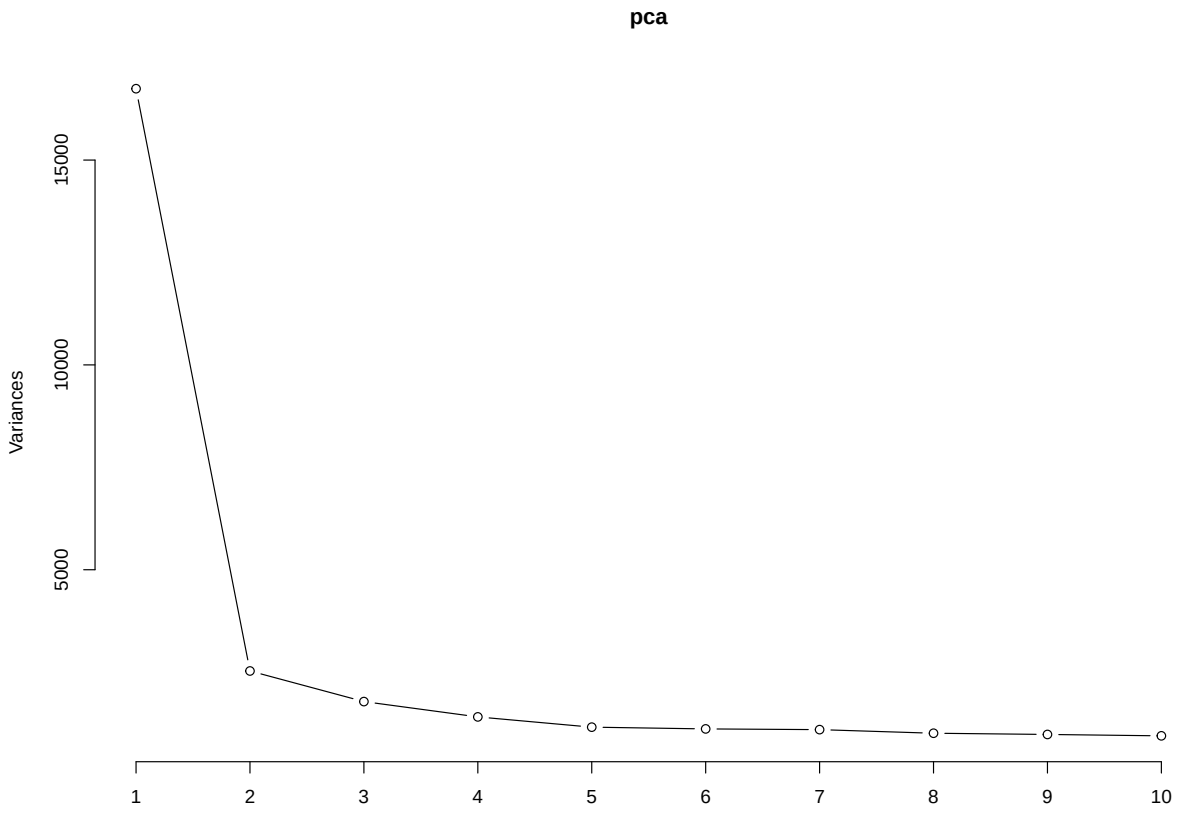
Exploratory PCA

```
# PCA analysis
EDASeq::plotPCA(RawExprMtx, k=3, labels=FALSE, col=col, pch=20)
legend("right", legend = names(colorMap), col = colorMap, pch = 19, bty = "n",cex = .7)
```



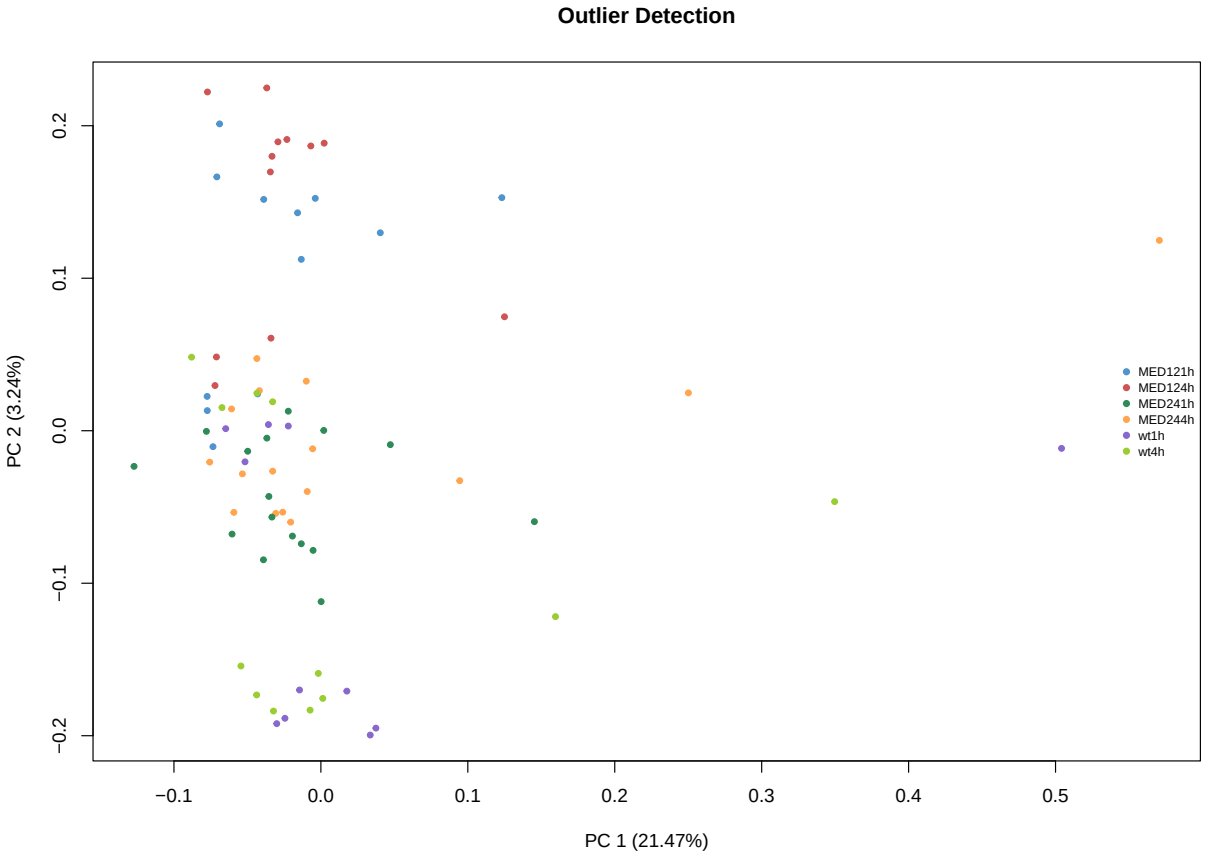
To have a first glance at global expression dynamics and determine the sources of variance, Principal Component Analysis (PCA) was performed on the log_{1p}-transformed data, using Genotype and treatment timepoints labels.Principal Component 2 and 3 clearly separate wt samples from the MED12 KO and MED24 KO groups, within the wt group, 1h (purple) and 4h (light green) cluster closely together, suggesting minimal transcriptomic differences over time in wild-type conditions. In contrast, dispersion inside the MED12 and MED24 groups is wider. Suggesting significant transcriptomic variability compared to the wild-type baseline.

```r
#PCA screen plot
pca = prcomp(t(log1p(RawExprMtx)))
screeplot(pca, type = c("lines"))
```



The scree plot displays a sharp, classic “elbow” transition at PC2, confirming that PC1 captures the dominant fraction of global dataset variance, however, PC2 and PC3 are better depicting de variability between the biological and temporal groups.

```r
EDASeq::plotPCA(RawExprMtx, k=2, labels=FALSE, pch=20, col = col, main = "Outlier Detection")
legend("right", legend = names(colorMap), col = colorMap, pch = 19, bty = "n",cex = .7)
```



A visual assessment of sample distribution via PCA was performed, and sample-level filtering was done in the following code to samples with datapoints deviating by more than three standard deviations from the PC1 or PC2.

```
# Coordinate extraction
pca_scores <- as.data.frame(pca$x)

# function to detect outliers (3 standard deviations)
detect_outliers <- function(column) {
  abs(column - mean(column)) > (3 * sd(column))
}

outliers_PC1 <- rownames(pca_scores)[detect_outliers(pca_scores$PC1)]
outliers_PC2 <- rownames(pca_scores)[detect_outliers(pca_scores$PC2)]

# Inspection
outliers_names <- unique(c(outliers_PC1, outliers_PC2))
outliers = match(outliers_names, colnames(RawExprMtx))
```

The diagnostic plots validate that further analysis should only proceed on the raw matrices directly. These exploratory analysis establishes two evident exigencies: first, the exclusion of sample outliers, and the application of a robust normalization framework to adress artifacts and composition biases seen in the MD plots.

GC content gene length

In high-throughput RNA-sequencing data workflows, two primary sources of within-sample, gene-specific technical bias variation are gene length and guanine-cytosine (GC) content. To isolate true biological variation from these confounding technical artifacts, downstream normalization strategies must be employed. These algorithms require the physical properties of each genetic feature as input covariates which are retrieved in the following code block, executing the database queries required to isolate stable Ensembl identifiers and fetch the corresponding length and nucleotide composition from the human reference genome to adress this.

```
#substring ENSMBL IDs
ensid = substr(as.character(rowRanges(data)$gene_id), 1, 15) #remove version of the gene
rowData(data)$ensid = ensid

#for each gene compute GC content and Length retrieval using human genome database
gc = getGeneLengthAndGCContent(id = ensid, org = "hg38", mode = "org.db")
```

Matrices filtering

Unannotated genetic features, duplicated entries, and genes with missing structural metadata (such as missing GC-content or length values) lack the necessary information for further normalization and biological interpretation.

Prior to normalization "filterByExpr" from the edgeR package, algorithm from the edgeR package was applied to establish a minimum threshold of reads across replicates, lowcount genes that would only artificially increase variance.

```
gc   = gc[!is.na(gc[, 2]), ] #remove genes without information
data = data[rowData(data)$ensid %in% rownames(gc), ] #keep on database only genes with information
#data and gc have the same genes now

#remove duplicates
dup       = duplicated(rownames(gc)) #list of true and false
gc        = gc[!dup, ] #remove duplicates from gc
data      = data[!dup, ] #remove duplicates from data

raw_count =  assay(data) #raw count matrix filtered

rownames(raw_count) = rownames(gc) #same names of genes (in rows)

# Setting groups and filtering by "filterByExpr"
keep = filterByExpr(raw_count, group = ExperimentalInfo$Groups)
expr_filtered_w_outliers = raw_count[keep, ] #extract genes where keep=TRUE
expr_filtered <- expr_filtered_w_outliers[, -outliers]

# Keep data and gc in sync with the filtered count matrix
ExprSel = data[rowData(data)$ensid %in% rownames(expr_filtered), ]
ExprSel <- ExprSel[, !(colnames(ExprSel) %in% outliers_names)]
gc_sel   = gc[rownames(expr_filtered), ]

#Delete outliers from experimental info
ExperimentalInfo <- ExperimentalInfo[!(rownames(ExperimentalInfo) %in% outliers_names), ]
#to compare data before and after filtering
```

```r
PlotDF = rbind(
  data.frame(sums = rowSums(raw_count),      status = "Before"),
  data.frame(sums = rowSums(expr_filtered), status = "After")
)

par(mfrow = c(1, 2))

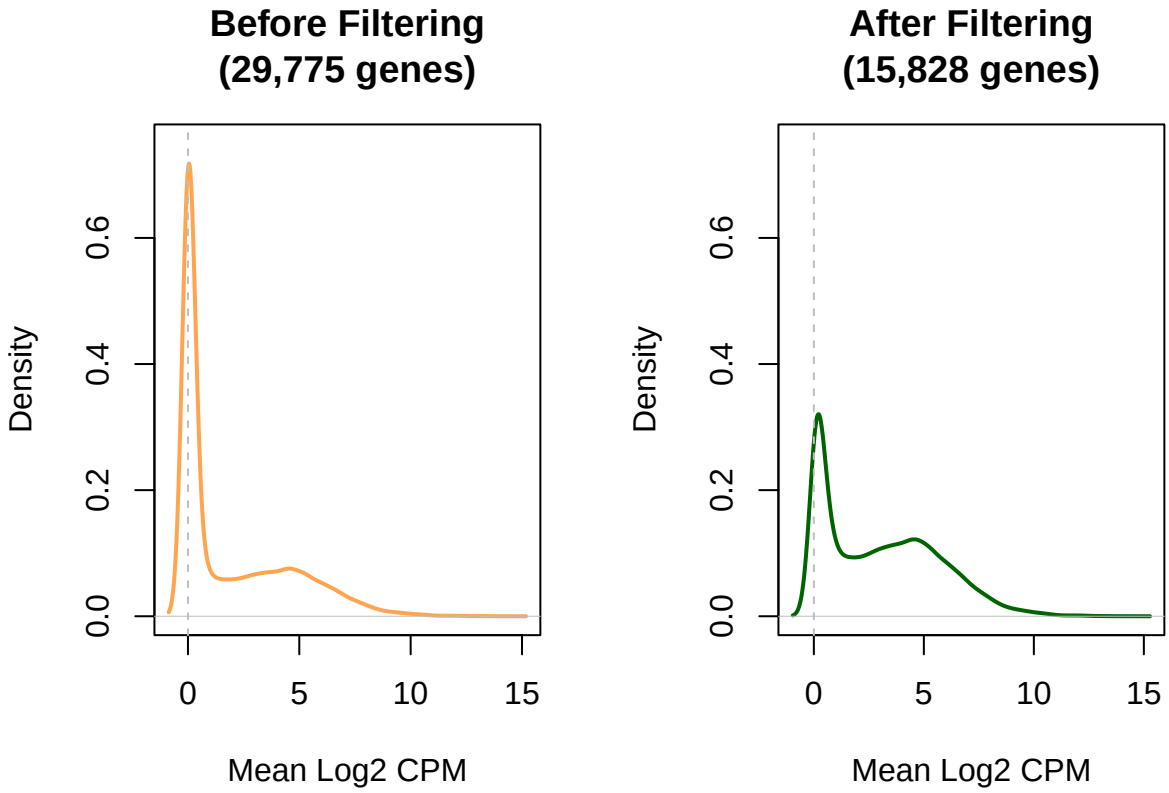
# Define a shared Y-axis
shared_ylim <- c(0, 0.75)

# Plot BEFORE filtering
log_cpm_before <- log2(edgeR::cpm(raw_count) + 1)
mean_log_cpm_before <- rowMeans(log_cpm_before)

plot(density(mean_log_cpm_before),
     main = "Before Filtering\n(29,775 genes)",
     xlab = "Mean Log2 CPM",
     ylab = "Density",
     col = "tan1", lwd = 2,
     ylim = shared_ylim)
abline(v = 0, col = "gray", lty = 2)

# Plot AFTER filtering
log_cpm_after <- log2(edgeR::cpm(expr_filtered) + 1)
mean_log_cpm_after <- rowMeans(log_cpm_after)

plot(density(mean_log_cpm_after),
     main = "After Filtering\n(15,828 genes)",
     xlab = "Mean Log2 CPM",
     ylab = "Density",
     col = "darkgreen", lwd = 2,
     ylim = shared_ylim)
abline(v = 0, col = "gray", lty = 2)
```



The figure shows the density distribution of gene expression levels before and after low-expression filtering. The X-axis represent a standardized measure of gene expression leves in a $\log_2$ scale, the Y-axis shows the proportion of genes that share a specific expression value.

Before filtering, the density plot shows a massive peak around 0, indicating that a large number of the 29.775 genes have very low or zero expression level, which represent the background noise. After filtering, the dataset was reduced to 15828 genes, resulting in a more balanced distribution, suitable for differential expression analysis.

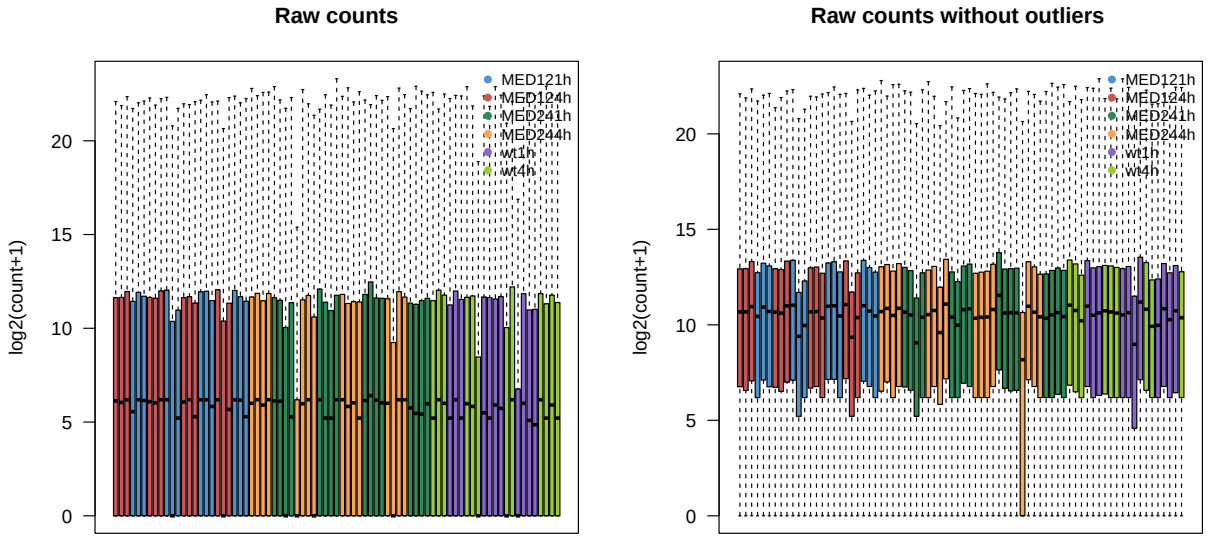
The density plots confirm that the filtering pipeline successfully separated uninformative technical background noise from true biological signal.

```
#boxplot for outliers removal
# objective: to verify the successful removal of low-quality sample outliers by
#comparing total sequencing depth across the dataset

par(mfrow = c(1, 2))
boxplot(log2(raw_count + 1), outline = FALSE, las = 2,
        col = col, ylab = "log2(count+1)",
        main = "Raw counts", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

boxplot(log2(expr_filtered + 1), outline = FALSE, las = 2,
        col = col, ylab = "log2(count+1)",
        main = "Raw counts without outliers ", xaxt = "n")
legend("topright", legend = names(colorMap),
```

```
        col = colorMap, pch = 19, bty = "n", cex = 0.8)
```



The figure above shows the impact on the dataset of removing three outliers. The X-axis represent biological samples, while the Y-axis represent the log-transformed gene expression level, calculated as $\log_2(Count+1)$.

During the PCA exploratory analysis, outliers were statistically detected and defined as samples whose coordinates exceeded three standard deviations from the mean along either the first or second principal component. This mathematical threshold flagged three samples as low-quality sequencing libraries, which were subsequently excluded from downstream analysis.

The PCA-filtering successfully removed the extreme outliers, from the plot in the right panel is visible that two samples continue to exhibit distinct structural profiles in the cleaned dataset, with higher variance and a larger proportion of low-expressed genes. Despite these reasons, these 2 samples were retained in the dataset because their median expression level remains aligned and they did not exceed the mathematical threshold of 3 standard deviation.

Normalization

Since sequencing datasets are heavily influenced by the technical process of sequencing itself, the primary aim of normalization is to remove systematic technical variation between samples, so it will be possible to compare their true biological differences.

Technical biases in RNA-Seq architectures operate across two distinct dimensions: intra-sample (within-lane) and inter-sample (between-lane) variables, each requiring targeted algorithmic correction to preserve data integrity.

Within-lane artifacts distort the relative expression profiles of genomic features within a single discrete library. These variations are predominantly driven by the physical architecture of the transcripts, specifically transcript length and nucleotide composition. During cDNA fragmentation, longer transcripts naturally yield a higher absolute mass of fragments, artificially inflating their raw read counts relative to shorter transcripts of identical cellular abundance. Concurrently, local Guanine-Cytosine (GC) content skews data distribution due to thermal stability differentials during polymerase chain reaction (PCR) amplification and cluster generation, leading to the systematic underrepresentation of genomic regions with extreme GC ratios.

Inter-Sample distortions occur across different samples, compromising the comparability of the dataset. They are primarily driven by variations in sequencing depth (total library size differences between lanes) and compositional bias (highly expressed master transcripts depleting the sequencing resources available for low-abundance genes).

Conversely, between-lane distortions compromise comparability across distinct sequencing tracks and independent biological samples. These variations are primarily driven by discrepancies in absolute sequencing depth, resulting from unequal sample pooling or variable cluster densities on the flow cell.
Furthermore, inter-sample analysis is highly susceptible to RNA compositional bias, a phenomenon where a restricted cohort of highly expressed transcripts monopolizes a disproportionate share of the total sequencing throughput. This over representation mathematically depletes the remaining read budget available for low-abundance transcripts, inducing false observations of global downregulation in unnormalized datasets.

To establish an optimal scaling and artifact-correction architecture capable of standardizing these distribution variances across all 76 independent libraries, two parallel exploratory normalization tracks were executed and evaluated.

To determine an optimal scaling and artifact-correction architecture able to standardize the variance for our dataset across all 76 independent libraries, two parallel exploratory normalization tracks were executed and evaluated.

Initialization of data structure SeqExpressionSet

To prepare the data for within-lane normalization and diagnostic tracking, EDASeq package was used to create a SeqExpressionSet object, which integrates the filtered raw count matrix with both sample-level metadata (phenoData) and gene-specific biophysical covariates (featureData), satisfying a mandatory requirement for normalization algorithm.

```
library(EDASeq)

set = newSeqExpressionSet(
          expr_filtered, #filtered counts
          phenoData = AnnotatedDataFrame( #metadata, information on samples
                        data.frame(
                        conditions = factor(1:ncol(expr_filtered)),
                        row.names = colnames(expr_filtered))),
          featureData = AnnotatedDataFrame( #info on gene length and gc content
                        data.frame(
                          gc = gc_sel[, 2],
                          l = gc_sel[, 1])))
```

Between lane Normalization

We compute four distinct between-lane scaling methods via the edgeR ecosystem to evaluate global variance adjustments. Ultimately, the Trimmed Mean of M-values (TMM) approach will be prioritized as our definitive mathematical baseline for downstream differential expression testing due to its superior capacity to handle compositional bias.

```
#  edgeR-based methods
lbs_tmm = calcNormFactors(expr_filtered, method = "TMM")
tmm     = cpm(expr_filtered, lib.size = lbs_tmm * colSums(expr_filtered))

lbs_rle = calcNormFactors(expr_filtered, method = "RLE")
rle_mat = cpm(expr_filtered, lib.size = lbs_rle * colSums(expr_filtered))
```

```r
lbs_uq  = calcNormFactors(expr_filtered, method = "upperquartile")
uq      = cpm(expr_filtered, lib.size = lbs_uq  * colSums(expr_filtered))

tc      = cpm(expr_filtered)   # total count (CPM)
```

Within lane Normalization

Concurrently, we evaluated a multi-stage normalization framework using the EDASeq package. This track departs from the global approach of edgeR by isolating sample-specific biophysical artifacts (within-lane bias) and coupling them sequentially with global cross-lane adjustments (between-lane bias).

```r
# --- EDASeq within-lane (GC correction) + upper-quartile between-lane ---
wt      = withinLaneNormalization(set, "gc", which = "upper", offset = TRUE)
set_uq  = betweenLaneNormalization(wt,  which = "upper", offset = TRUE)
set_full = betweenLaneNormalization(wt, which = "full",  offset = TRUE)

full    = set_full@assayData$normalizedCounts
uq_eda  = set_uq@assayData$normalizedCounts
```

Global Expression Distribution Diagnostics Across Six Normalization States

To visualize how different statistical corrections reshape the distribution of reads in the dataset, we generated sequential $\log_2(Count + 1)$ boxplots across six successive stages of data processing. This comparative layout maps the distribution of the transcriptomic data from its raw, unnormalized state through four individual scaling methods: Total Count (CPM), Relative Log Expression (RLE), Trimmed Mean of M-values (TMM), and Upper Quartile (UQ). In the end, it shows the result of the EDASeq full normalization pipeline.

By looking at these panels, we can evaluate how effectively each method balance the distribution and stabilize sample medians, without distorting the underlying biological variance.

```r
#comparative normalization diagnostic boxplots
#objective: To evaluate, compare, and select the optimal normalization method
#by monitoring how different strategies align global gene expression across all samples.
par(mfrow = c(6, 1), mar = c(2, 4, 2, 1))

# Raw Counts:Exposes baseline technical variations.
boxplot(log2(expr_filtered + 1), outline = FALSE, las = 2,
        col = col, ylab = "log2(count+1)",
        main = "Raw counts", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

# Total Count (CPM - Counts Per Million): A basic library-size scaling method.
boxplot(log2(tc   + 1), outline = FALSE, las = 2,
        col = col, ylab = "log2(CPM+1)",
        main = "Total count (CPM)", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

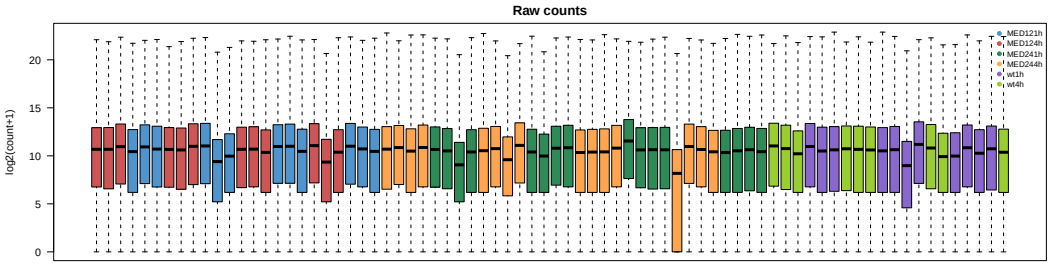
# RLE (Relative Log Expression Normalization): Forces the samples to align
#based on a median gene pseudo-reference.
```

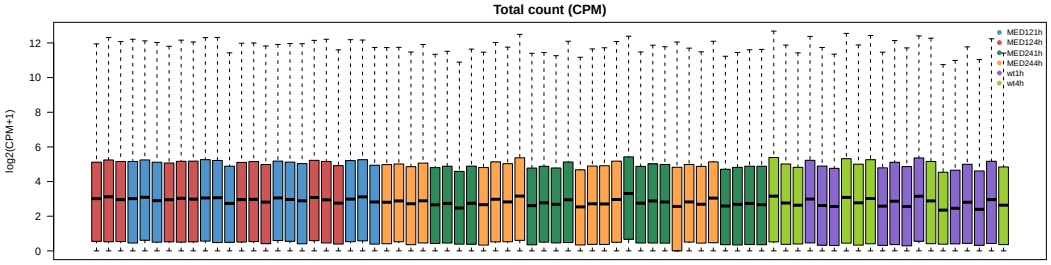
```r
boxplot(log2(rle_mat + 1), outline = FALSE, las = 2,
        col = col, ylab = "log2(CPM+1)",
        main = "RLE", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

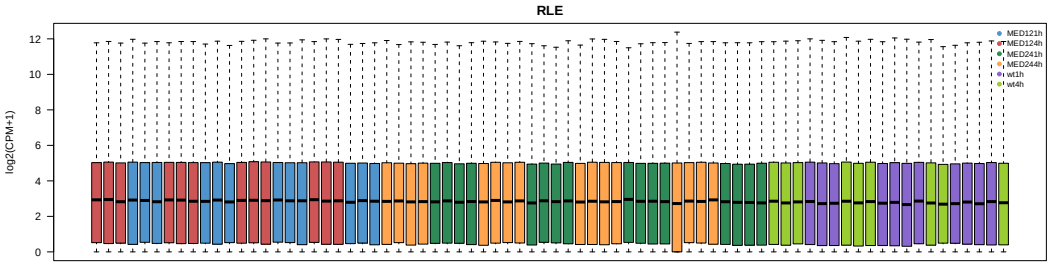
# TMM (Trimmed Mean of M-values): The standard edgeR normalization.
#It handles composition bias by trimming away highly variable and lowly expressed genes
#before calculating scaling factors.
boxplot(log2(tmm  + 1), outline = FALSE, las = 2,
        col = col, ylab = "log2(CPM+1)",
        main = "TMM", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

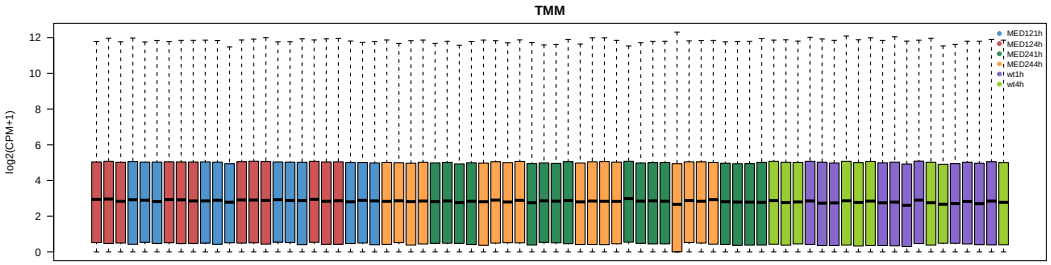
# Upper Quartile: Scales counts based on the 75th percentile of expression
boxplot(log2(uq   + 1), outline = FALSE, las = 2,
        col = col, ylab = "log2(CPM+1)",
        main = "Upper quartile", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

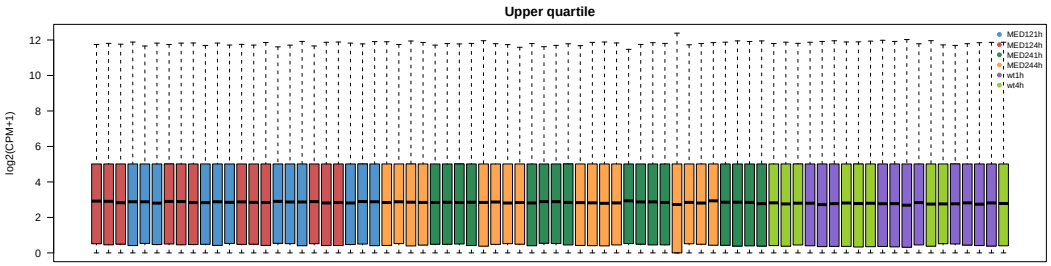
# EDASeq full (GC + Between-Lane Normalization): It explicitly corrects
#for GC-content and gene length biases
boxplot(log2(full + 1), outline = FALSE, las = 2,
        col = col, ylab = "log2(normalizedCounts+1)",
        main = "EDASeq full (GC + between-lane)", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)
```

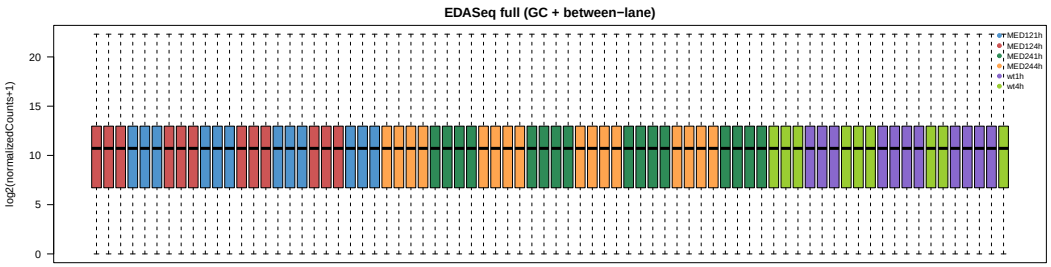

Raw counts
log2(count+1)
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


Total count (CPM)
log2(CPM+1)
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


RLE
log2(CPM+1)
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


TMM
log2(CPM+1)
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


Upper quartile
log2(CPM+1)
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


EDASeq full (GC + between−lane)
log2(normalizedCounts+1)
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h

```
par(mfrow = c(1, 1))
```

The initial Raw Counts panel highlights substantial technical noise, showing visible discrepancies in box heights, interquartile ranges, and median alignments across the libraries due to variations in sequencing depth. In the next plot, Total Count (CPM) normalization scales the data globally to correct for library size, which narrows the variance but still leaves noticeable unevenness in the medians and spreads between different biological groups.

The advanced methods (RLE, Upper Quartile, and EDASeq full) show significant improvements over raw counts. They successfully flatten the dataset and bring the samples into a much closer global alignment. However, despite these corrections, subtle box-to-box fluctuations and minor shifts in the horizontal baseline still persist between the experimental genotypes and timepoints.

The most optimal and uniform alignment is achieved via TMM (Trimmed Mean of M-values) normalization. This method effectively neutralizes both sequencing depth discrepancies and sample-specific composition biases without obscuring or over-smoothing the underlying biological variation.

Relative Log Expression (RLE)

To critically evaluate how effectively different normalization strategies eliminate systematic technical biases, we generated Relative Log Expression (RLE) plots sequentially across six distinct computational states.

By centering each gene's expression around its median across all samples, the RLE profiles provide a direct diagnostic assessment of sample-wise deviations.

```
par(mfrow = c(6, 1), mar = c(2, 4, 2, 1))

# objective: To benchmark the effectiveness of various normalization strategies by
#evaluating their ability to eliminate compositional bias and unwanted technical variance.

#Raw counts:Exposes significant baseline batch effects and library size variations.
EDASeq::plotRLE(expr_filtered, outline = FALSE, las = 2, col = col,
                ylab = "RLE", main = "Raw counts", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

#Total count (CPM): Simple scaling pulls the sample medians closer to zero
EDASeq::plotRLE(tc,      outline = FALSE, las = 2, col = col,
                ylab = "RLE", main = "Total count (CPM)", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

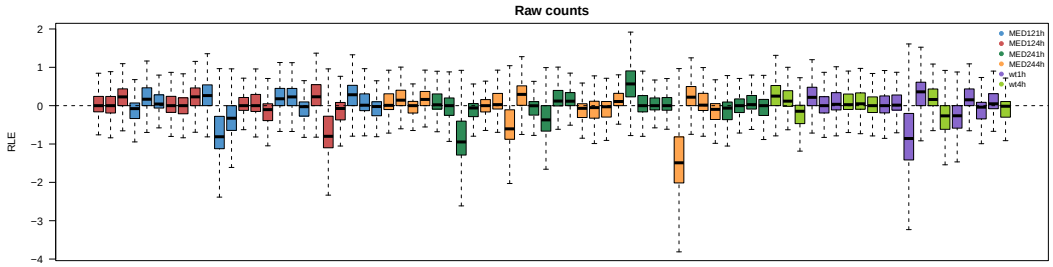
#TMM (Trimmed Mean of M-values): Significantly improves the data.
#By trimming extreme values.
EDASeq::plotRLE(tmm,     outline = FALSE, las = 2, col = col,
                ylab = "RLE", main = "TMM", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

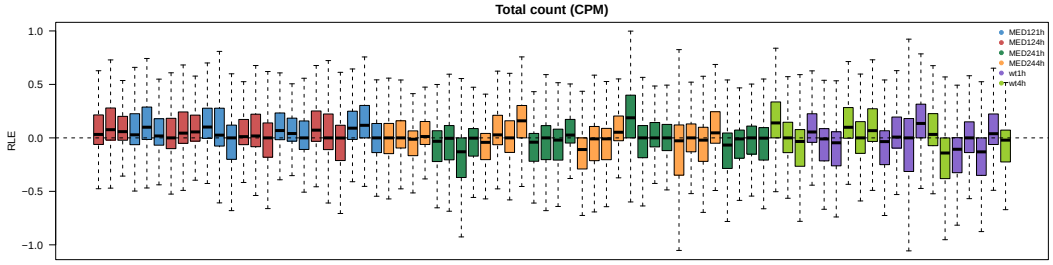
#RLE (Relative Log Expression Matrix): Explicitly centers the gene deviations
#across samples.
EDASeq::plotRLE(rle_mat, outline = FALSE, las = 2, col = col,
                ylab = "RLE", main = "RLE", xaxt = "n")
legend("topright", legend = names(colorMap),
```

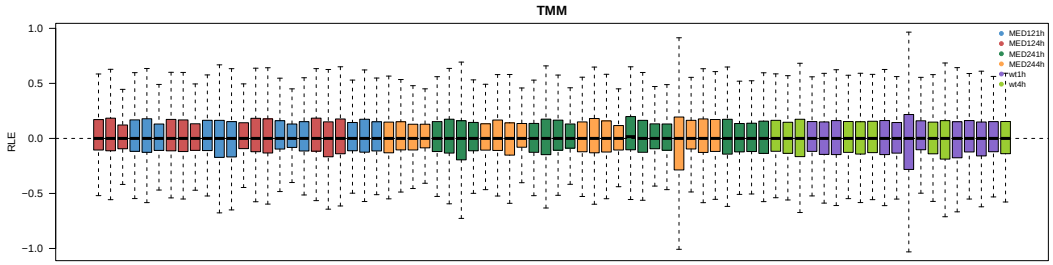
```
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

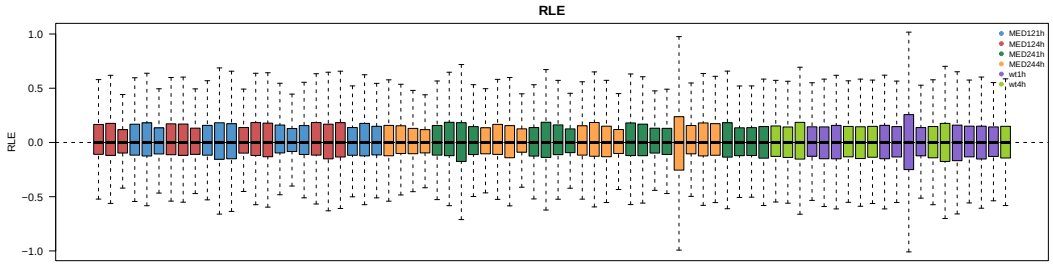
#Upper quartile: Achieves a highly stable centering around zero by scaling
#to the 75th percentile.
EDASeq::plotRLE(uq,      outline = FALSE, las = 2, col = col,
                ylab = "RLE", main = "Upper quartile", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)

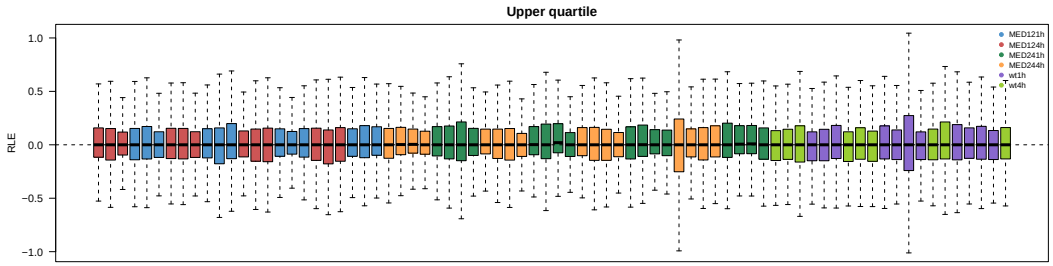
# EDASeq full (GC + Between-lane): It handles within-lane feature
EDASeq::plotRLE(full,    outline = FALSE, las = 2, col = col,
                ylab = "RLE", main = "EDASeq full", xaxt = "n")
legend("topright", legend = names(colorMap),
       col = colorMap, pch = 19, bty = "n", cex = 0.8)
```

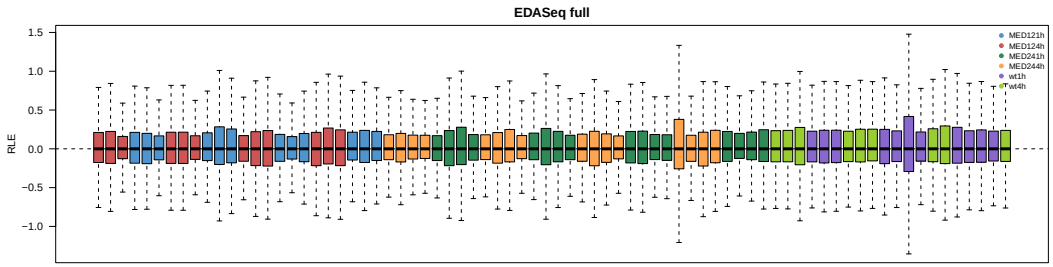

Raw counts
RLE
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


Total count (CPM)
RLE
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


TMM
RLE
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


RLE
RLE
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


Upper quartile
RLE
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h


EDASeq full
RLE
MED121h
MED124h
MED241h
MED244h
wt1h
wt4h

```
par(mfrow = c(1, 1))
```

Effect of normalization on library size

To evaluate how the Trimmed Mean of M-values (TMM) workflow i,pacts on our data, the distribution statistics of the library sizes was analyzed across all samples before and after transformation.

Prior to normalization, the raw library sizes exhibit massive mathematical scales, with a median sequencing depth exceeding 258 million reads per sample and a high degree of dispersion with a Coefficient of Variation (CV) of 22.45%.

To address these discrepancies, the TMM algorithm calculates sample-specific scaling factors based on the relative expression of a trimmed baseline of moderately expressed, non-differentially expressed genes. Rather than altering the raw sequencing counts, the algorithm adjust the library sizes to compute effective library sizes.

The global inter-sample variability is minimized, with the Coefficient of Variation dropping from 22.45% down to 12.24%. By compressing this technical noise while preserving the biological differences between the 1h and 4h timepoints, the dataset is successfully stabilized against compositional and depth-driven biases.

```
library(knitr)


lib_sizes_before = colSums(expr_filtered)
lib_sizes_after = colSums(tmm)


stats_before <- summary(lib_sizes_before)
stats_after  <- summary(lib_sizes_after)

cv_before <- sd(lib_sizes_before) / mean(lib_sizes_before) * 100
cv_after  <- sd(lib_sizes_after) / mean(lib_sizes_after) * 100

df_stats <- data.frame(
  Metric = c(
    "Minimum Library Size",
    "1st Quartile",
    "Median",
    "Mean",
    "3rd Quartile",
    "Maximum Library Size",
    "Coefficient of Variation (CV)"
  ),
  `Before Normalization (Raw)` = c(
    format(round(as.numeric(stats_before[1])), big.mark=","),
    format(round(as.numeric(stats_before[2])), big.mark=","),
    format(round(as.numeric(stats_before[3])), big.mark=","),
    format(round(as.numeric(stats_before[4])), big.mark=","),
    format(round(as.numeric(stats_before[5])), big.mark=","),
    format(round(as.numeric(stats_before[6])), big.mark=","),
    paste0(format(round(cv_before, 2), nsmall=2), "%")
  ),
  `After Normalization (TMM)` = c(
    format(round(as.numeric(stats_after[1])), big.mark=","),
```

```
    format(round(as.numeric(stats_after[2])), big.mark=","),
    format(round(as.numeric(stats_after[3])), big.mark=","),
    format(round(as.numeric(stats_after[4])), big.mark=","),
    format(round(as.numeric(stats_after[5])), big.mark=","),
    format(round(as.numeric(stats_after[6])), big.mark=","),
    paste0(format(round(cv_after, 2), nsmall=2), "%")
  ),
  check.names = FALSE
)

kable(df_stats,
      align = c("l", "c", "c"),
      caption = "Library size distribution statistics across the RNA-Seq samples.")
```

Table 1: Library size distribution statistics across the RNA-Seq samples.

Metric	Before Normalization (Raw)	After Normalization (TMM)
Minimum Library Size	58,559,060	776,876
1st Quartile	230,356,094	909,888
Median	258,600,053	993,307
Mean	248,615,327	1,007,330
3rd Quartile	285,690,530	1,095,692
Maximum Library Size	334,424,837	1,304,622
Coefficient of Variation (CV)	22.45%	12.24%

GC Content Bias Profiling

To assess how nucleotide composition impacts our sequencing data, we evaluated the relationship between gene-wise GC content and read counts both before and after applying a within-lane normalization framework.

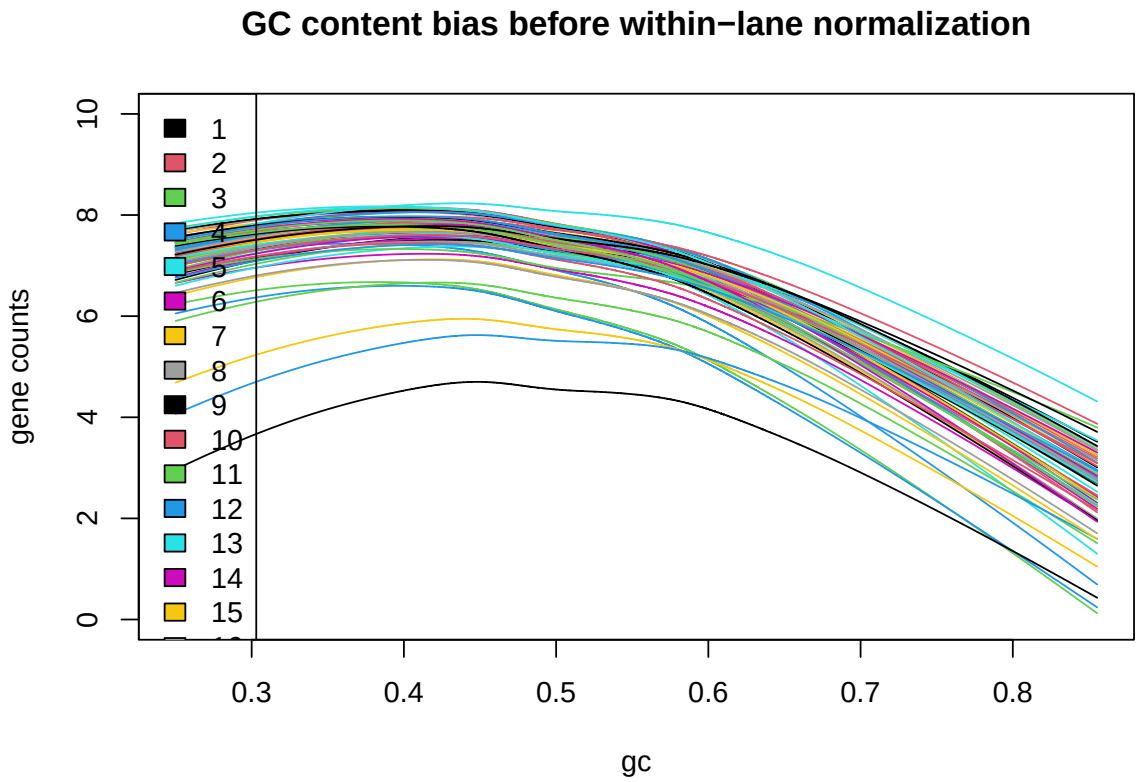
In the not normalized state, the diagnostics reveal a non-linear bias where read counts peak for genes with average GC fractions but drop off sharply as GC content increases. This curve is a classic signature of PCR amplification issues.

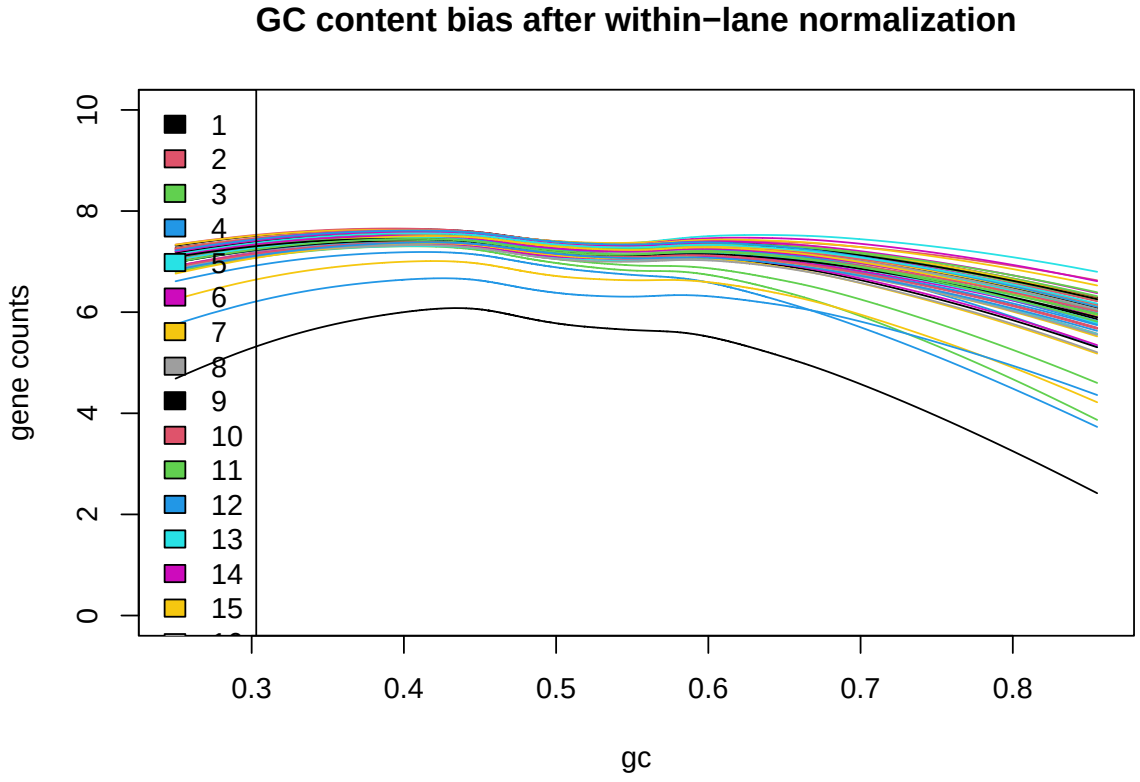
The post-normalization bias plots illustrate the performance of the correction framework. By flattening these curves across the entire GC spectrum, the normalization process decouples gene's physical nucleotide structure from its measured expression level. This targeted correction ensures that the remaining expression values reflect the transcript abundance rather than sequence-dependent technical artifacts.

```
#within-lane bias evaluation
#objective: to verify the effectiveness of normalization
par(mfrow = c(2, 1))

EDASeq::biasPlot(set, "gc", ylim = c(0, 10), log = TRUE,
                 main = "GC content bias before within-lane normalization")

EDASeq::biasPlot(set_uq, "gc", ylim = c(0, 10), log = TRUE,
                 main = "GC content bias after within-lane normalization")
```

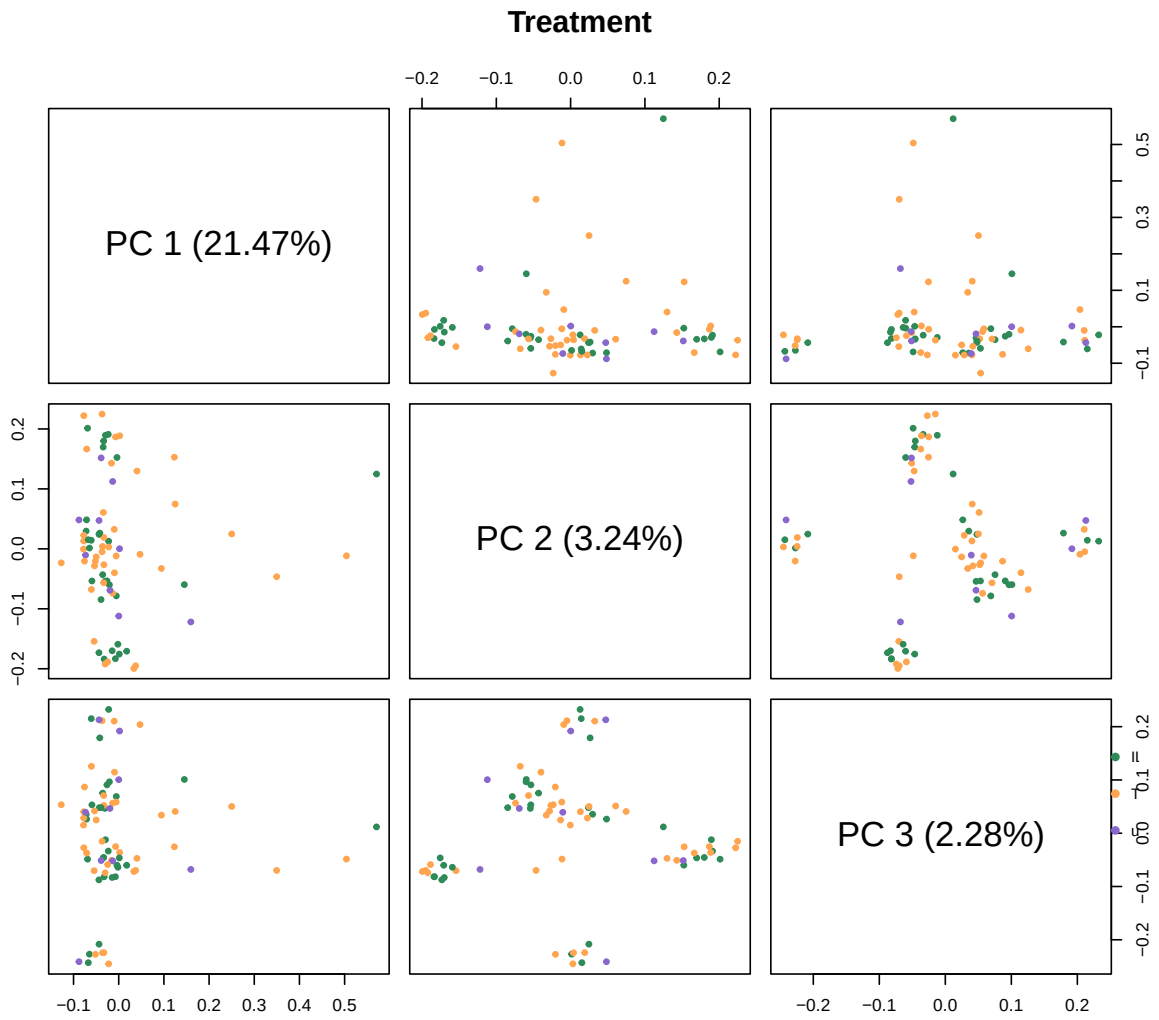

GC content bias before within−lane normalization
gene counts
gc
0
2
4
6
8
10
0.3
0.4
0.5
0.6
0.7
0.8
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15


GC content bias after within−lane normalization
gene counts
gc
0
2
4
6
8
10
0.3
0.4
0.5
0.6
0.7
0.8
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

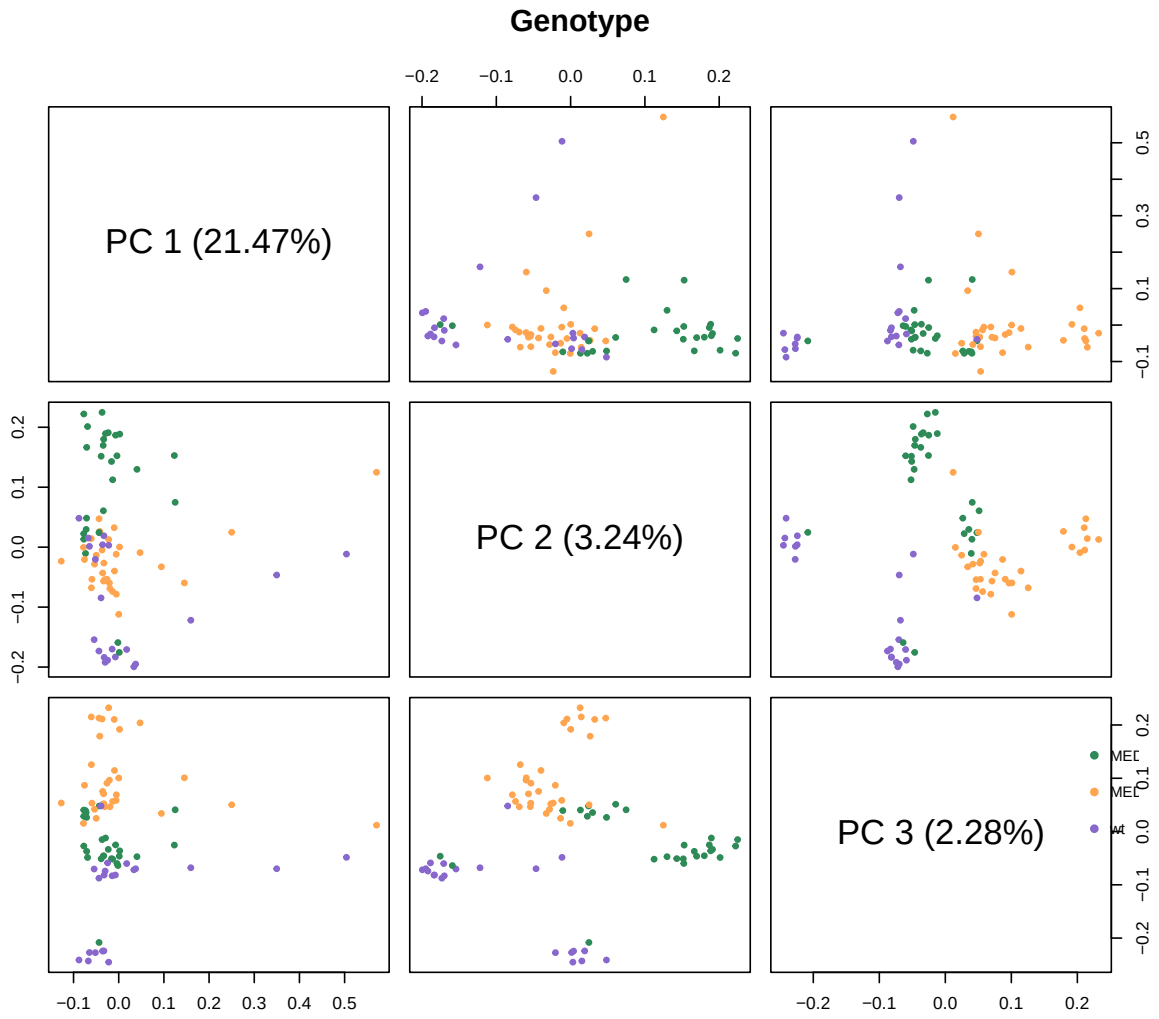
```r
#to evaluate the clustering of the dominant sources of biological variance by
#overlaying different experimental factors (Treatment vs. Genotype) onto the PCA space


par(mfrow = c(2, 1))
#by treatment
COLORS =  c("seagreen4", "tan1", "mediumpurple3")

stimu  = as.factor(substr(ExperimentalInfo$stimulation,1,3))
EDASeq::plotPCA(RawExprMtx, k=3, labels=FALSE, col=COLORS[stimu], pch=20, main = "Treatment")
legend("right", legend = c("IL1", "TNF", "uns"), col = COLORS, pch = 19, bty = "n",cex = .6)
```




```r
#by genotype
geno = as.factor(ExperimentalInfo$genotype)
EDASeq::plotPCA(RawExprMtx, k=3, labels=FALSE, col= COLORS[geno], pch=20, main = "Genotype")
legend("right", legend = c( "MED12", "MED24", "wt"),col = COLORS, pch = 19, bty = "n",cex = .6)
```




```
par(mfrow = c(1, 1))
```

Correlation heatmap (TMM)

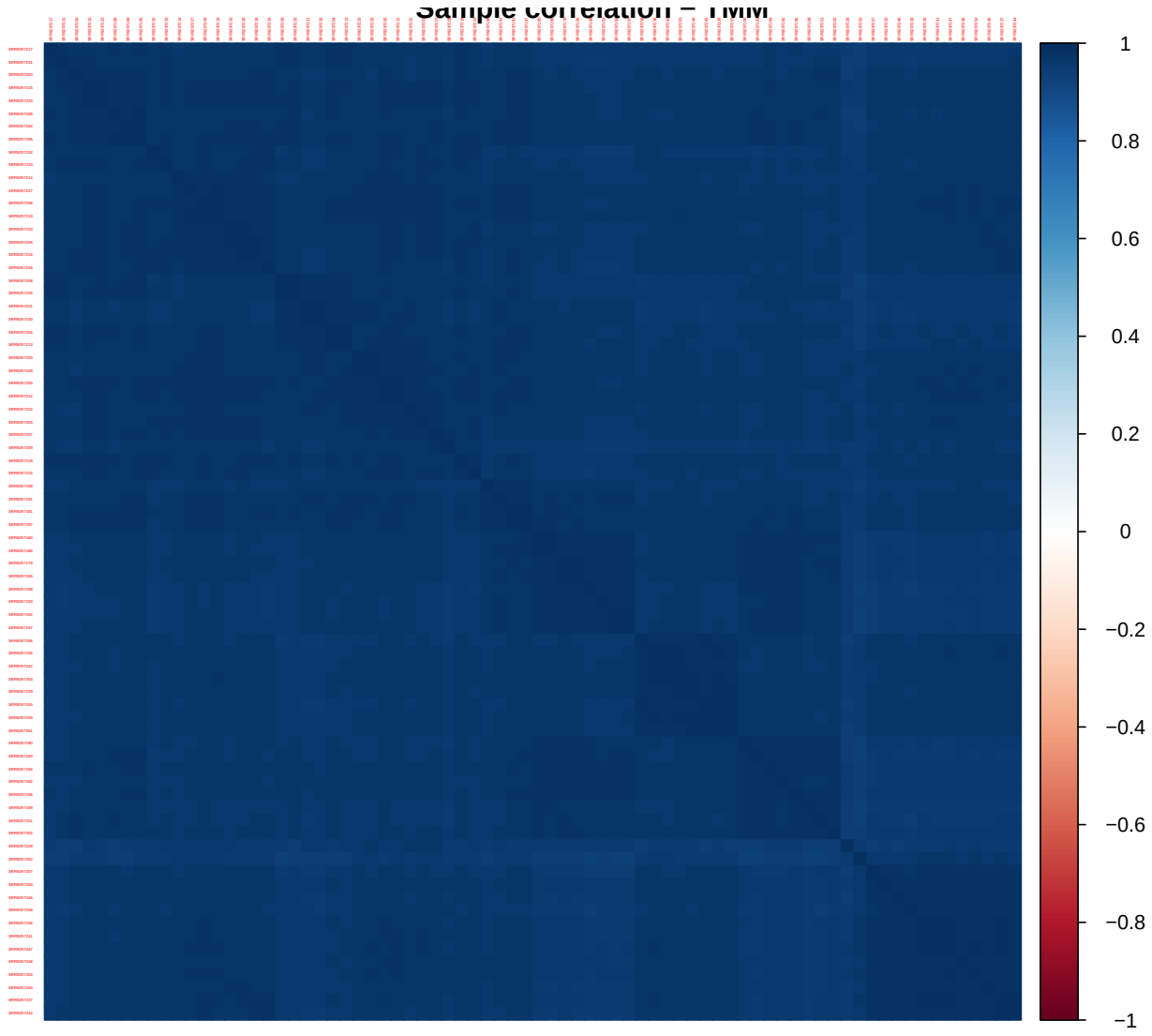
The TMM-normalized correlation heatmap displays the sample-to-sample Pearson correlation coefficients across the entire filtered dataset.

The global matrix shows high and uniform correlation values (clustering tightly between 0.98 and 1.00), confirming that the TMM normalization workflow established a stable baseline across all sequenced libraries. The extreme homogeneity in the map demonstrates that technical noise has been suppressed.

Since the global transcriptome is highly stable, the specialized statistical models utilized in downstream Differential Expression Analysis (such as edgeR or DESeq2) can accurately estimate gene-wise dispersions and can isolate the precise, targeted transcriptional shifts driving the differences between the 1h and 4h experimental conditions.

```
#sample-to-sample correlation heatmap
#objective: To assess global similarity, reproducibility, and clustering patterns across
#all samples by computing and visualizing their pairwise Pearson correlation coefficients.
```

```
m = cor(log1p(tmm))
corrplot(m, method = "color", order = "hclust", tl.cex = 0.2,
         main = "Sample correlation - TMM")
```



Differential Expression Analysis

To identify significantly differentially expressed genes (DEGs) across our experimental conditions (1h vs 4h across various genetic backgrounds) and to ensure the reproducibility of our results, differential expression analysis was performed using the intersection of significantly altered genes across three prominent Bioconductor packages: edgeR, deseq2 and NOISeq. In this way, we minimize package-specific false positives and isolate the Differentially Expressed Genes (DEGs) governing the p65 translocation response.

EdgeR

With edgeR Bioconductor package we implemented a differential expression test across the genotypes and timepoints using a generalized linear model framework.

To evaluate changes across time and genetic conditions, we combined genotype (MED12, MED24, WT) and timepoint (1h, 4h) factors into a single group variable. This parametrization was used to build an intercept-free design matrix. Following TMM normalization and empirical Bayes dispersion estimation, we applied a negative binomial generalized linear model to handle the biological variation across replicates. Then we defined explicit contrast vectors to isolate the temporal changes occurring between the 1-hour and 4-hour timepoints for each genotype separately.

```
library(edgeR)

#Model matrix design by groups
group = factor(paste(ExperimentalInfo$genotype,
                     ExperimentalInfo$timepoint, sep = "_")) #unified variables

design = model.matrix(~ 0 + group)

#Groups to expr_prox
ExprPro = ExprSel
ExprPro$Timepoints = as.factor(ExperimentalInfo$timepoint) #add column timepoint

dge = DGEList(counts = assay(ExprPro), group = group) #matrix with timepoints
dge = calcNormFactors(dge, method = "TMM") #add factors of normalization
dge = estimateDisp(dge, design) #how much genes varies between samples of the same group
FitEdger = glmFit(dge, design)

#names for defining the contrasts
con <- makeContrasts(
  MED12_4vs1 = groupMED12_4h - groupMED12_1h,
  MED24_4vs1 = groupMED24_4h - groupMED24_1h,
  WT_4vs1    = groupwt_4h    - groupwt_1h,
  levels = design
)

fit <- glmQLFit(dge, design)

lrt_M12 <- glmQLFTest(fit, contrast = con[, "MED12_4vs1"])
lrt_M24 <- glmQLFTest(fit, contrast = con[, "MED24_4vs1"])
lrt_WT  <- glmQLFTest(fit, contrast = con[, "WT_4vs1"])

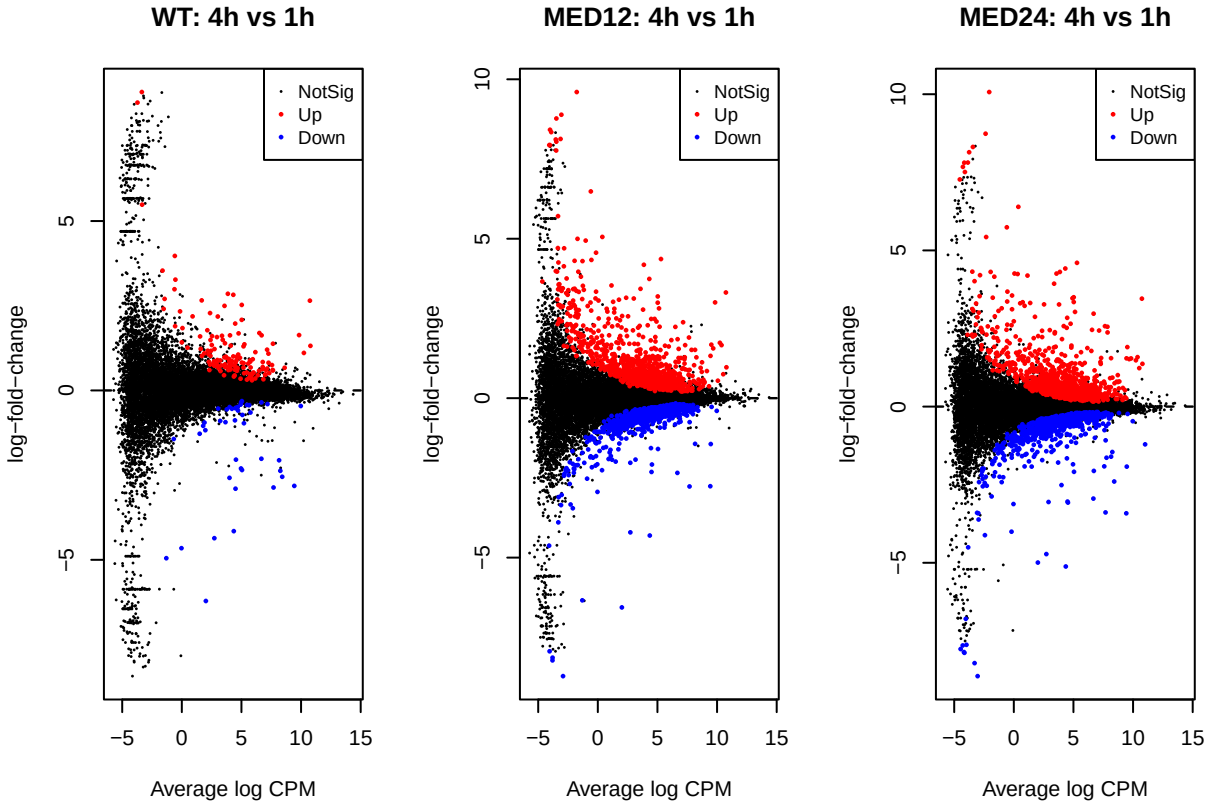
#resulting tables
MED12edger <- as.data.frame(topTags(lrt_M12, n = Inf)$table)
MED24edger <- as.data.frame(topTags(lrt_M24, n = Inf)$table)
WTedger    <- as.data.frame(topTags(lrt_WT, n = Inf)$table)


par(mfrow = c(1, 3))

#Wild Type
plotMD(lrt_WT, hl.col = c("red", "blue"), main = "WT: 4h vs 1h", cex = 0.5)
abline(h = 0, col = "black", lty = 2, lwd = 1.2)

#MED12
plotMD(lrt_M12, hl.col = c("red", "blue"), main = "MED12: 4h vs 1h", cex = 0.5)
abline(h = 0, col = "black", lty = 2, lwd = 1.2)
```

```r
#MED24
plotMD(lrt_M24, hl.col = c("red", "blue"), main = "MED24: 4h vs 1h", cex = 0.5)
abline(h = 0, col = "black", lty = 2, lwd = 1.2)
```




```r
par(mfrow = c(1, 1))
```

To visualize the distribution and magnitude of transcriptional changes occurring during the temporal transition (4h vs. 1h) under cytokine stimulation, we generated edgeR Mean-Difference (MD) plots across the three genetic backgrounds.

In the plots, the X-axis represents the baseline expression level (*Average* log CPM), while the Y-axis displays the biological effect size (log-fold-change). Statistically significant differentially expressed genes (FDR $<$ 0.05) are highlighted in red (up-regulated at 4h relative to 1h) and blue (down-regulated at 4h relative to 1h), while non-significant features remain black.

The wild type plot show that under normal conditions, transitioning from 1 hour to 4 hours causes little transcriptomic upheaval. We can visualize only a small portion of upregulated and downregulated genes, which indicates a stable minimal response over time.

In contrast, MED12 and MED24 plots show a massive increase in significant altered genes. These results show that deleting or mutating MED12 or MED24 breaks the cell's normal homostatic control between 1h and 4h, triggering a cascade of differential expressed genes. Furthermore, in these plots the upregulated genes show a larger cloud than the downregulated genes, which indicates that the temporal response in these mutants is dominated by transcriptional activation or a failure to repress genes at 4-hour mark.

DESeq2

```
#install.packages('ashr')
library(DESeq2)
#RawSummarized experiment with genes that survived the filtering and no outliers
ExprSel = data[rowData(data)$ensid %in% rownames(expr_filtered),
               !(colnames(data) %in% outliers_names)]
ExprPro = ExprSel
ExprPro$group = group #add timepoint+celltype to the data

#Analysis
dds = DESeqDataSet(ExprPro, design = ~ group)
dds = DESeq(dds)

#data normalized to compare with limma and edgeR
counts_norm = counts(dds, normalized = TRUE)


# MED12 (4h vs 1h)
DeseqRawM12 = results(dds,contrast = c("group", "MED12_4h", "MED12_1h"), alpha = 0.05)
DeseqShrunkM12 = as.data.frame(lfcShrink(dds,
                                         contrast = c("group", "MED12_4h", "MED12_1h"),
                                         type = "ashr"))


# MED24 (4h vs 1h)
DeseqRawM24 = results( dds, contrast = c("group", "MED24_4h", "MED24_1h"), alpha = 0.05)
DeseqShrunkM24 = as.data.frame(lfcShrink( dds,
                                          contrast = c("group", "MED24_4h", "MED24_1h"),
                                          type = "ashr" ))


# WT (4h vs 1h)
DeseqRawWt = results(dds,contrast = c("group", "wt_4h", "wt_1h"), alpha = 0.05 )
DeseqShrunkWt = as.data.frame(lfcShrink(dds,
                                        contrast = c("group", "wt_4h", "wt_1h"),
                                        type = "ashr"))
```

NOISeqbio

```
library(NOISeq)
ExprProx = ExprSel
# setting levels
levels  = data.frame(group = group)
rownames(levels) = colnames(assay(ExprProx))

NoiseqData = readData(data = assay(ExprProx), factors = levels)

invisible(capture.output({ #to hide output of those functions
NoiseqM12 = noiseqbio(NoiseqData,  norm = "tmm", factor = "group",
                      conditions = c("MED12_4h", "MED12_1h"))
```

```
NoiseqM24 = noiseqbio(NoiseqData, norm = "tmm", factor = "group",
                      conditions = c("MED24_4h", "MED24_1h"))
NoiseqWt = noiseqbio(NoiseqData, norm = "tmm", factor = "group",
                     conditions = c("wt_4h", "wt_1h"))
}))


NoiseqM12 = as.data.frame(NoiseqM12@results)
NoiseqM24 = as.data.frame(NoiseqM24@results)
NoiseqWt = as.data.frame(NoiseqWt@results)
```

Consensus DEGs

To ensure an unbiased comparison, we standardized the results from differential expressed algorithms: edgeR, DESeq2 and NOISeq. Since each software package utilizes its own nomenclature, statistical metrics and row-naming convention, we developed a preprocessing function (ClassificationDEA) to standardize the outputs across all genotypes.

```
#orders = EdgeR, Deseq, Noiseq

M12DEA = list(MED12edger, DeseqShrunkM12, NoiseqM12)
M24DEA = list(MED24edger, DeseqShrunkM24, NoiseqM24)
WtDEA = list(WTedger, DeseqShrunkWt, NoiseqWt)


ClassificationDEA <- function(x){

  #identifier cleansing
  rownames(x) <- substr(rownames(x), 1, 15)
  x <- x[!duplicated(rownames(x)), ]

  #statistical metric harmonization
  # FDR
  if("padj" %in% colnames(x)) x$p <- x$padj
  if("PValue" %in% colnames(x)) x$p <- x$PValue
  if("prob" %in% colnames(x)) x$p = 1 - x$prob

  # logFC
  if("log2FoldChange" %in% colnames(x)) x$logFC <- x$log2FoldChange
  if("log2FC" %in% colnames(x)) x$logFC <- x$log2FC

  # Classification
  x$sign <- "NS"
  x$sign[x$p < 0.05 & x$logFC > 0] <- "Up"
  x$sign[x$p < 0.05 & x$logFC < 0] <- "Down"

  # Binary Differential Expression
  x$DE <- 0
  x$DE[x$p < 0.05] <- 1

  # Odering by log
  x <- x[order(x$logFC, decreasing = TRUE), ]
```

```
  return(x)
}


AnnotatedWtDEA = lapply(WtDEA, ClassificationDEA )
AnnotatedM12DEA = lapply(M12DEA, ClassificationDEA )
AnnotatedM24DEA = lapply(M24DEA, ClassificationDEA )
```

Evaluation of Consensus Genes

To extract a list of high reliable differential expressed genes (DEG), we implemented a custom function (ConsensusDEA) which integrates the preprocessed outputs from edgeR, DESeq2 and NOISeq to identify robust genes that are independently validated across all three statistical frameworks.

Firstly, the function extracts the genes identified as DEG by each individual package and then it applies an intersection across the groups, extracting a list of 'robust genes'. To measure the degree of overlap between pairs of algorithms, the function calculates a pairwise Jaccard similarity coefficient matrix, which is computed as the size of the intersection of two genes list divided by the size of their union.

To verify that the tools agree not just on significance, but on the magnitude of change, the function builds an independent pairwise Pearson correlation matrix, which isolates the genes shared between any two methods, extracts their respective effect sizes ($logFC$), and evaluates their correlation.

Finally, the function scrutinizes the global consensus genes to verify their directional logic, with a matrix where a gene is retained only if its mathematical sign is perfectly consistent across all frameworks.

At the end, the final high-confidence matrix is sorted in descending order to prioritize top candidates for downstream pathway analysis.

```
library(ggplot2)
library(ggrepel)
library(ggplot2)
library(ggrepel)

ConsensusDEA <- function(list_matrices) {

  # Name assigning
  nombres_estandar <- c("edgeR", "DESeq2", "Noiseq")
  names(list_matrices) <- nombres_estandar[seq_along(list_matrices)]
  metodos <- names(list_matrices)

  # DE gene extraction
  genes_DE <- lapply(list_matrices, function(x) {
    rownames(x)[x$DE == 1 & !is.na(x$DE)]
  })

  # Intersection of shared genes (global consensus)
  RobustGenes <- Reduce(intersect, genes_DE)

  # Matriz Jaccard
  jaccard_mat <- sapply(metodos, function(a) {
    sapply(metodos, function(b) {
      u <- length(union(genes_DE[[a]], genes_DE[[b]]))
      if (u == 0) return(0)
```

```
    length(intersect(genes_DE[[a]], genes_DE[[b]])) / u
  })
})

jaccard_mat <- matrix(jaccard_mat, nrow = length(metodos),
                      dimnames = list(metodos, metodos))

# Labeling genes
TotalSummary <- do.call(rbind, lapply(list_matrices, function(x) {
  data.frame(
    Up = sum(x$sign == "Up", na.rm = TRUE),
    Down = sum(x$sign == "Down", na.rm = TRUE),
    TotalDE = sum(x$DE == 1, na.rm = TRUE)
  )
}))
rownames(TotalSummary) <- metodos

# Initialize an empty correlation matrix with method names
cor_matrix <- matrix(NA, nrow = length(metodos), ncol = length(metodos),
                     dimnames = list(metodos, metodos))

# Calculate pairwise correlations
for (m1 in metodos) {
  for (m2 in metodos) {
    #Find genes shared between just these two specific methods
    shared_pair_genes <- intersect(genes_DE[[m1]], genes_DE[[m2]])

    #Only calculate correlation if they share MORE THAN 2 genes
    if (length(shared_pair_genes) > 2) {
      logfc1 <- list_matrices[[m1]][shared_pair_genes, "logFC"]
      logfc2 <- list_matrices[[m2]][shared_pair_genes, "logFC"]

      #Calculate the correlation for this specific pair
      cor_matrix[m1, m2] <- cor(logfc1, logfc2, use = "pairwise.complete.obs")
    } else if (m1 == m2 && length(genes_DE[[m1]]) > 1) {
      # Keep perfect self-correlation on the diagonal if the method has genes
      cor_matrix[m1, m2] <- 1.0
    }
  }
}



# Robust LogFC tracking
robust_logFC_out <- NULL
if (length(RobustGenes) > 0) {
  logfc_matrix <- do.call(cbind, lapply(list_matrices, function(x) x[RobustGenes, "logFC"]))
  colnames(logfc_matrix) <- metodos
  rownames(logfc_matrix) <- RobustGenes

  if (length(list_matrices) == 1) {
    consistent_sign <- rep(TRUE, nrow(logfc_matrix))
  } else {
```

```
      consistent_sign <- apply(logfc_matrix, 1, function(row) all(sign(row) == sign(row[1])))
    }

    filtered_logfc <- logfc_matrix[consistent_sign, , drop = FALSE]

    if (nrow(filtered_logfc) > 0) {
      mean_logfc <- rowMeans(filtered_logfc)
      robust_logFC_out <- as.matrix(sort(mean_logfc, decreasing = TRUE))
      colnames(robust_logFC_out) <- "Mean_LogFC"
    }
  }

  return(list(
    common_genes = RobustGenes,
    summary = TotalSummary,
    jaccard_matrix = jaccard_mat,
    correlation_logFC = cor_matrix,
    robust_logFC_matrix = robust_logFC_out
  ))
}



# Run the analysis
ResultadosWT  = ConsensusDEA(AnnotatedWtDEA)
ResultadosM12 = ConsensusDEA(AnnotatedM12DEA)
ResultadosM24 = ConsensusDEA(AnnotatedM24DEA)
```

Volcano plots

```
library(patchwork)
library(ggplot2)
library(ggrepel)

## volcano plots
Volcano = function(df, name) {
  top = df[order(df$p), ][1:20, ]
  ggplot(df, aes(x = logFC, y = -log10(p), color = sign)) +geom_point(alpha = 0.4) +
    scale_color_manual(values = c("Down"="blue", "Up"="red", "NS"="black")) +
    geom_text_repel(data = top, aes(label = rownames(top)), size = 3) +
    labs(title = name) + theme_classic()
}


Libraries= c("EdgeR", "Deseq", "Noiseq")

VolcanoWT = mapply(Volcano, AnnotatedWtDEA, Libraries)
VolcanoM12 = mapply(Volcano, AnnotatedM12DEA, Libraries)
VolcanoM24 = mapply(Volcano, AnnotatedM24DEA, Libraries)

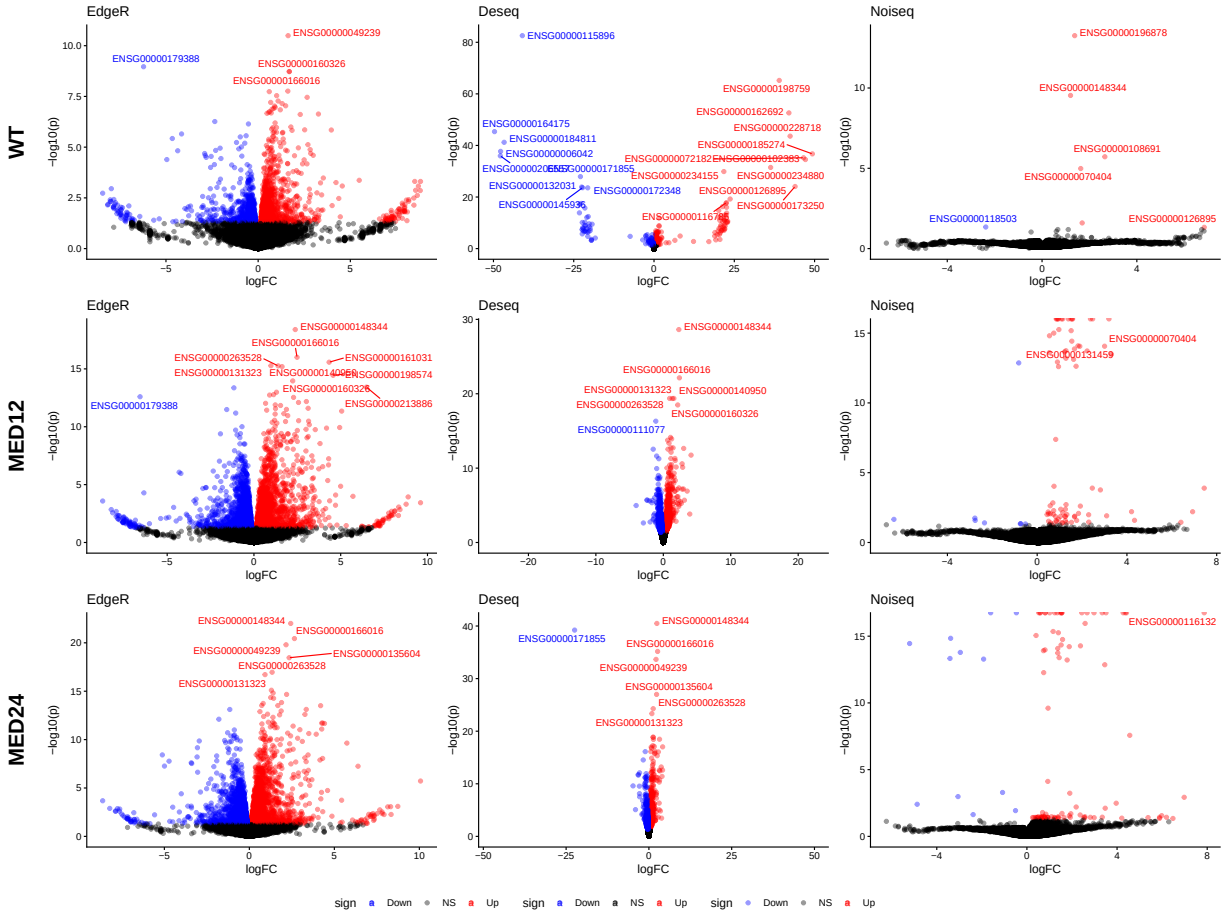

#Improved helper function for row labels
```

```r
etiqueta_fila = function(texto) {
  ggplot() +
    annotate("text", x = 0.5, y = 0.5, label = texto,
             angle = 90, size = 7, fontface = "bold") +
    coord_cartesian(xlim = c(0, 1), ylim = c(0, 1), expand = FALSE) +
    theme_void()
}

#Generate plot lists
plots_WT  = mapply(Volcano, AnnotatedWtDEA, Libraries, SIMPLIFY = FALSE)
plots_M12 = mapply(Volcano, AnnotatedM12DEA, Libraries, SIMPLIFY = FALSE)
plots_M24 = mapply(Volcano, AnnotatedM24DEA, Libraries, SIMPLIFY = FALSE)

#Combine into a single flat list
all_plots = c(
  list(etiqueta_fila("WT")),    plots_WT,
  list(etiqueta_fila("MED12")), plots_M12,
  list(etiqueta_fila("MED24")), plots_M24
)

# Assemble with collected legends
# The '&' operator applies the theme to ALL subplots in the patchwork
wrap_plots(all_plots, ncol = 4, widths = c(0.15, 1, 1, 1)) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom")
```



To evaluate both the statistical significance and the biological effect size of the temporal transition (4h vs. 1h), we generated a 3×3 matrix of volcano plots mapping each genetic background against our three differential expression pipelines. Each individual plot maps the statistical significance of a gene on the y-axis (calculated as $-\log_{10}(p\text{-value})$) against its biological effect size on the x-axis ($\log_2$-fold change). Red and blue dots represent respectively genes that are up-regulated and down-regulated at 4h respect to 1h. Black dots represent non-significative genes.

When evaluating the performance of the three pipelines horizontally across the software columns, we can see different algorithm behaviour. The edgeR framework demonstrates a highly classical, balanced distribution, capturing a symmetrical plot of both up-regulated and down-regulated transcripts across a smooth significance gradient. In contrast, DESeq2 exhibits a high sensitivity to low count noise in the wild-type libraries, mapping highly variable transcripts to extreme fold-change values on the horizontal axis, yet shrinking the high-confidence candidates into a tight vertical spire in the mutant groups. Meanwhile, NOISeq operates as the most conservative framework of the three. Because it relies on empirical, non-parametric probability testing rather than standard parametric modeling, its significance scales remain considerably lower, completely ignoring minor background fluctuations to isolate only the most extreme biological signals.

Looking across the biological rows reveals an unambiguous and highly reproducible cellular phenotype. In the wild-type control group, the transition from 1 hour to 4 hours is characterized by extreme transcriptomic stability, with both edgeR and NOISeq showing that the vast majority of genes remain completely unaltered. However, the moment MED12 or MED24 are deleted, this baseline homeostatic control breaks entirely. Both mutant strains trigger an explosive, genome-wide surge in differential gene expression. Crucially, this temporal response is heavily asymmetric, dominated by a massive cloud of highly significant, up-regulated genes. This consistent rightward skew indicates that the loss of either mediator subunit results in a widespread failure to repress transcripts or an uncoupled activation of gene expression over time.

When evaluating the transcriptomic response to p65 translocation during the 1h to 4h temporal transition, a visual discrepancy was observed between the differential expression pipelines. Specifically, the wild-type (WT) condition analyzed via the DESeq pipeline exhibited an anomalous volcano plot distribution, with a high density of features clustered at extreme effect sizes (logFC $\approx \pm 25$ and ± 50). This could be due a mathematical artifact arising from low-count or zero-count genes.

Summary table

```
library(kableExtra)

unificar = function(res) {
  target_methods <- c("edgeR", "DESeq2", "Noiseq")

  #align the General Summary block
  summary_block <- matrix(0, nrow = 3, ncol = 3,
                          dimnames = list(target_methods, c("Up", "Down", "TotalDE")))
  common_sum <- intersect(rownames(res$summary), target_methods)
  summary_block[common_sum, ] <- as.matrix(res$summary[common_sum, ])

  #align the Jaccard block
  jaccard_block <- matrix(NA, nrow = 3, ncol = 3,
                          dimnames = list(target_methods, target_methods))
  # If it exists, map the matching rows/columns over
  if(!is.null(res$jaccard_matrix)) {
    common_jac <- intersect(rownames(res$jaccard_matrix), target_methods)
    jaccard_block[common_jac, common_jac] <- res$jaccard_matrix[common_jac, common_jac]
  }

  #align the Correlation block
  cor_block <- matrix(NA, nrow = 3, ncol = 3,
                      dimnames = list(target_methods, target_methods))
  if(!is.null(res$correlation_logFC)) {
    common_cor <- intersect(rownames(res$correlation_logFC), target_methods)
    cor_block[common_cor, common_cor] <- res$correlation_logFC[common_cor, common_cor]
  }

  #combine blocks together
  cbind(summary_block, jaccard_block, cor_block)
}
```

```
TablaFinal = rbind(
  unificar(ResultadosWT),
  unificar(ResultadosM12),
  unificar(ResultadosM24)
)

# Render table
TablaFinal %>%
  kable(
    caption = "Concordance and DEG Profiling",
    booktabs = TRUE,
    digits = 3,
    col.names = c("Up", "Down", "Total",
                  "EdgeR", "Deseq", "Noiseq",
                  "EdgeR", "Deseq", "Noiseq")
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE,
    font_size = 10
  ) %>%
  add_header_above(c(" " = 1, "General summary" = 3,
                     "Similitude (Jaccard)" = 3, "Correlation" = 3)) %>%
  pack_rows("WILD TYPE", 1, 3) %>%
  pack_rows("MED12 KO", 4, 6) %>%
  pack_rows("MED24 KO", 7, 9)
```

Table 2: Concordance and DEG Profiling

	General summary			Similitude (Jaccard)			Correlation		
	Up	Down	Total	EdgeR	Deseq	Noiseq	EdgeR	Deseq	Noiseq
WILD TYPE									
edgeR	1138	796	1934	1.000	0.126	0.004	1.000	0.782	0.992
DESeq2	217	99	316	0.126	1.000	0.022	0.782	1.000	0.879
Noiseq	6	1	7	0.004	0.022	1.000	0.992	0.879	1.000
MED12 KO									
edgeR	2051	1899	3950	1.000	0.400	0.022	1.000	0.941	0.990
DESeq2	827	752	1579	0.400	1.000	0.051	0.941	1.000	0.905
Noiseq	77	8	85	0.022	0.051	1.000	0.990	0.905	1.000
MED24 KO									
edgeR	2022	2041	4063	1.000	0.425	0.026	1.000	0.963	0.987
DESeq2	962	768	1730	0.425	1.000	0.056	0.963	1.000	0.969
Noiseq	95	12	107	0.026	0.056	1.000	0.987	0.969	1.000

Inter-Algorithmic Performance Assessment

The comparative analysis reveals distinct variations in the detection sensitivity of expression analysis (DEA) frameworks between the edgeR, DESeq2, and NOISeq libraries, and across experimental conditions.

Across all genetic backgrounds, edgeR demonstrates the highest sensitivity, consistently yielding the largest number of Deferentially Expressed Genes, followed by DESeq2, whichconsistently acts as a more conservative

parametric capturing a narrower subset of genes. On the other hand NOISeq proves to be an extremely conservative algorithm, identifying significantly fewer genes, even showing nearzero detection in the Wild Type analysis, but with increasing yield in MED12 and MED24.

This diference in gene number detection is further displayed in the Jaccard similarity coefficients, This disparity in sensitivity results in a relatively low exact gene identity overlap, as evidenced by Jaccard similarity coefficients that peak at only 0.425 between edgeR and DESeq2 in the MED24 KO gene subset, and drop to near-zero when comparing NOISeq to any of the other two parametric methods.

However, the correlation coefficients of the LogFold Change in the list of shared genes thorughout methodologies consistently approach linearity, with the edgeR and DESeq2 relationship from 0.782 in WT to 0.963 in MED24 KO. Despite the different metodological approaches from parametric algorithms and NOISeq, the correlation align almost seamlessly between them keeping coefficients above 0.879 across all conditions.

Clustering

Only robust genes identified intersected by edgeR and DESeq2 were retained. NOISeq was excluded from the following pipeline due to lack of sensitivity, which failed to output a number of informative features big enough.

To evaluate whether the robust gene expression variability can reliably capture the differences between experimental timepoints, we implemented an exploratory Principal Component Analysis to visually identify clusters among biological groups. Also a pipeline was developed to compare unsupervised clustering algorithms (Hierarchical Clustering and K-means) using both the normalized matrix with only the robust genes from each biological group and its corresponding PCA dimensional space. The performance was validated using Silhouette widths and classification accuracy metrics (ARI) using the true biological timepoint labels as reference.

```
##CLUSTERING


library(cluster)
library(factoextra)
library(mclust)


ResultadosWT = ConsensusDEA(AnnotatedWtDEA[-3]) #we are not using Noiseq for wildtype,
ResultadosM12 = ConsensusDEA(AnnotatedM12DEA[-3])
ResultadosM24 = ConsensusDEA(AnnotatedM24DEA[-3])#reasons shown above



# subsetting tmm matrix
tmmWt = tmm[ResultadosWT$common_genes,ExperimentalInfo$genotype == "wt"]
tmmMED12 = tmm[ResultadosM12$common_genes, ExperimentalInfo$genotype == "MED12" ]
tmmMED24= tmm[ResultadosM24$common_genes, ExperimentalInfo$genotype == "MED24" ]


par(mfrow = c(1,3))

COLORSTime = c("plum3", "skyblue")


tiempo_WT    = as.factor(ExperimentalInfo$timepoint[ExperimentalInfo$genotype == "wt"])
```

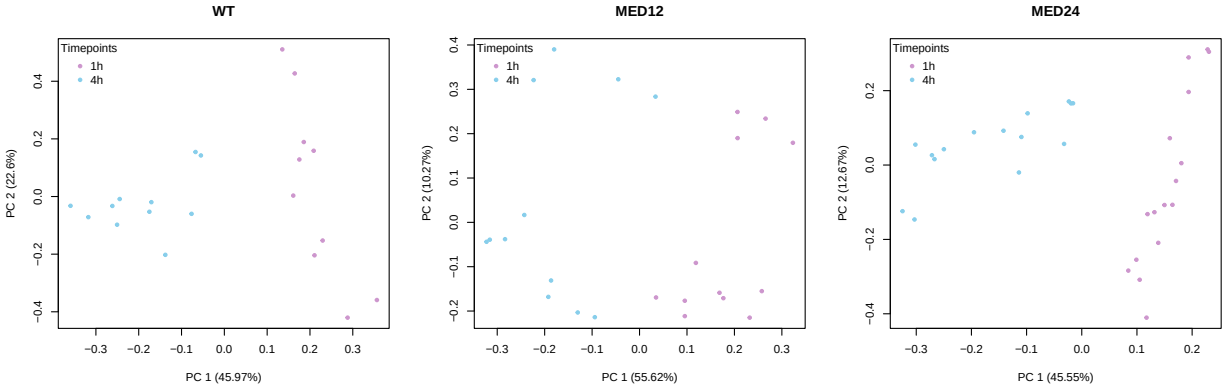
```
tiempo_MED12 = as.factor(ExperimentalInfo$timepoint[ExperimentalInfo$genotype == "MED12"])
tiempo_MED24 = as.factor(ExperimentalInfo$timepoint[ExperimentalInfo$genotype == "MED24"])

## evaluation of PCA projection between timepoints on robust genes

EDASeq::plotPCA(tmmWt, k=2, labels=FALSE, col=COLORSTime[tiempo_WT], pch=20, main = "WT")
legend("topleft", legend=levels(tiempo_WT), col=COLORSTime, pch=20, bty="n", title="Timepoints")

EDASeq::plotPCA(tmmMED12, k=2, labels=FALSE, col=COLORSTime[tiempo_MED12], pch=20, main = "MED12")
legend("topleft", legend=levels(tiempo_MED12), col=COLORSTime, pch=20, bty="n", title="Timepoints")

EDASeq::plotPCA(tmmMED24, k=2, labels=FALSE, col=COLORSTime[tiempo_MED24], pch=20, main = "MED24")
legend("topleft", legend=levels(tiempo_MED24), col=COLORSTime, pch=20, bty="n", title="Timepoints")
```



The Principal Component Analysis confirms that when stratifying the data by genotype, the temporal expression variability between the 1 hour and 4 hour timepoints comes out. In all genotype groups the 1st PC acts as the primary axis of separation, capturing 45% to 55% of the total variance.

```
truth = as.factor(ExperimentalInfo$timepoint)
LabelsWT = as.factor(ExperimentalInfo$timepoint[ExperimentalInfo$genotype == "wt"])
LabelsMED12 = as.factor(ExperimentalInfo$timepoint[ExperimentalInfo$genotype == "MED12"])
LabelsMED24 = as.factor(ExperimentalInfo$timepoint[ExperimentalInfo$genotype == "MED24"])


ClustersPCA = function(x, k, truth){

  # PCA
  pcaRobustG = prcomp(t(log1p(x)), scale. = TRUE)


  #the components are either 20 or the total number if its lower than 20
  n_pcs = min(20, ncol(pcaRobustG$x))
  pca_scores = pcaRobustG$x[,1:n_pcs, drop = FALSE]

  #PCA distance
  dist_pca = dist(pca_scores, method = "euclidean")

  #hierarchical PCA
  hc_pca = hclust(dist_pca, method = "complete")
  gr_hc_pca = cutree(hc_pca, k = k)
```

```r
  # Kmeans PCA
  set.seed(123)
  km_pca = kmeans(pca_scores, centers = k, nstart = 25)
  gr_km_pca = km_pca$cluster

  # Silhouette PCA
  sil_hc_pca = silhouette(gr_hc_pca, dist_pca)
  sil_km_pca = silhouette(gr_km_pca, dist_pca)

  # no PCA

  mat = t(log1p(x))
  mat = t(scale(t(mat))) #scaling
  dist_raw = dist(mat, method = "euclidean")

  # hierarchical raw
  hc_raw = hclust(dist_raw, method = "complete")
  gr_hc_raw = cutree(hc_raw, k = k)

  # Kmeans raw
  set.seed(123)
  km_raw = kmeans(mat, centers = k, nstart = 25)
  gr_km_raw = km_raw$cluster

  # Silhouette raw
  sil_hc_raw = silhouette(gr_hc_raw, dist_raw)
  sil_km_raw = silhouette(gr_km_raw, dist_raw)



  par(mfrow = c(2,2))

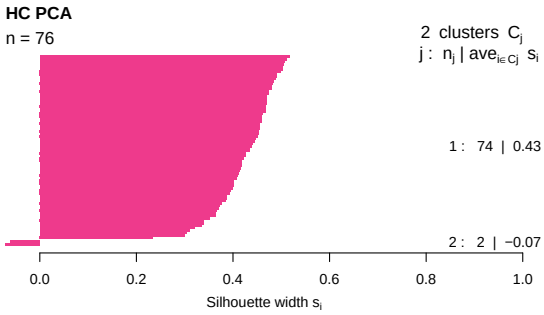
  plot(sil_hc_pca,main = "HC PCA",col = "violetred2",border = NA)
  plot(sil_km_pca,main = "Kmeans PCA",col = "orange",border = NA)
  plot(sil_hc_raw,main = "HC Raw",col = "steelblue",border = NA)
  plot(sil_km_raw,main = "Kmeans Raw",col = "darkgreen",border = NA)


  # ARI

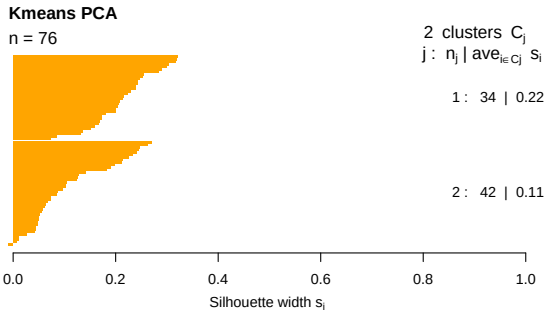
  cat("Kmeans PCA ARI:", adjustedRandIndex(truth, gr_km_pca), "\n")
  cat("HC PCA ARI:",adjustedRandIndex(truth, gr_hc_pca), "\n")
  cat("Kmeans Raw ARI:",adjustedRandIndex(truth, gr_km_raw), "\n")
  cat("HC Raw ARI:",adjustedRandIndex(truth, gr_hc_raw), "\n")
}


# Clustering functions

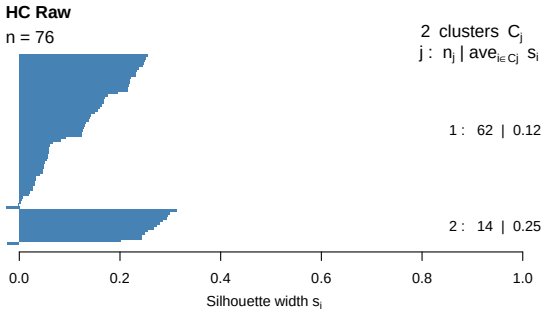
#Total normalized tmm
ClustersPCA(tmm,2,truth )
```



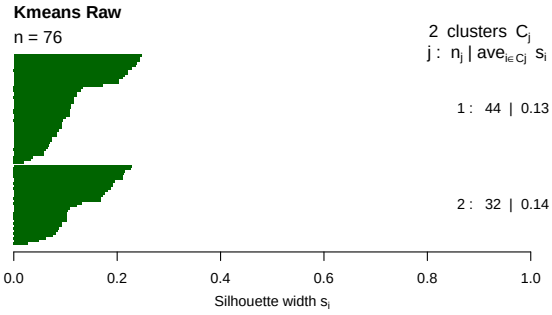
Average silhouette width : 0.41



Average silhouette width : 0.16



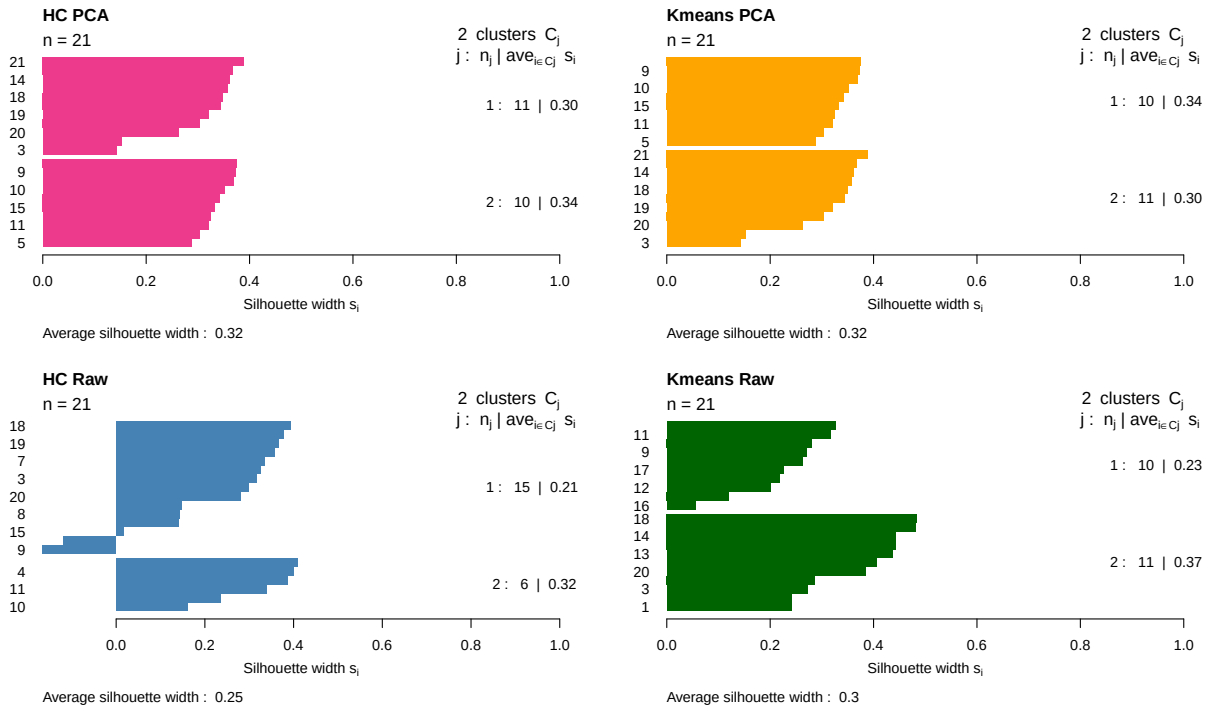
Average silhouette width : 0.14



Average silhouette width : 0.13

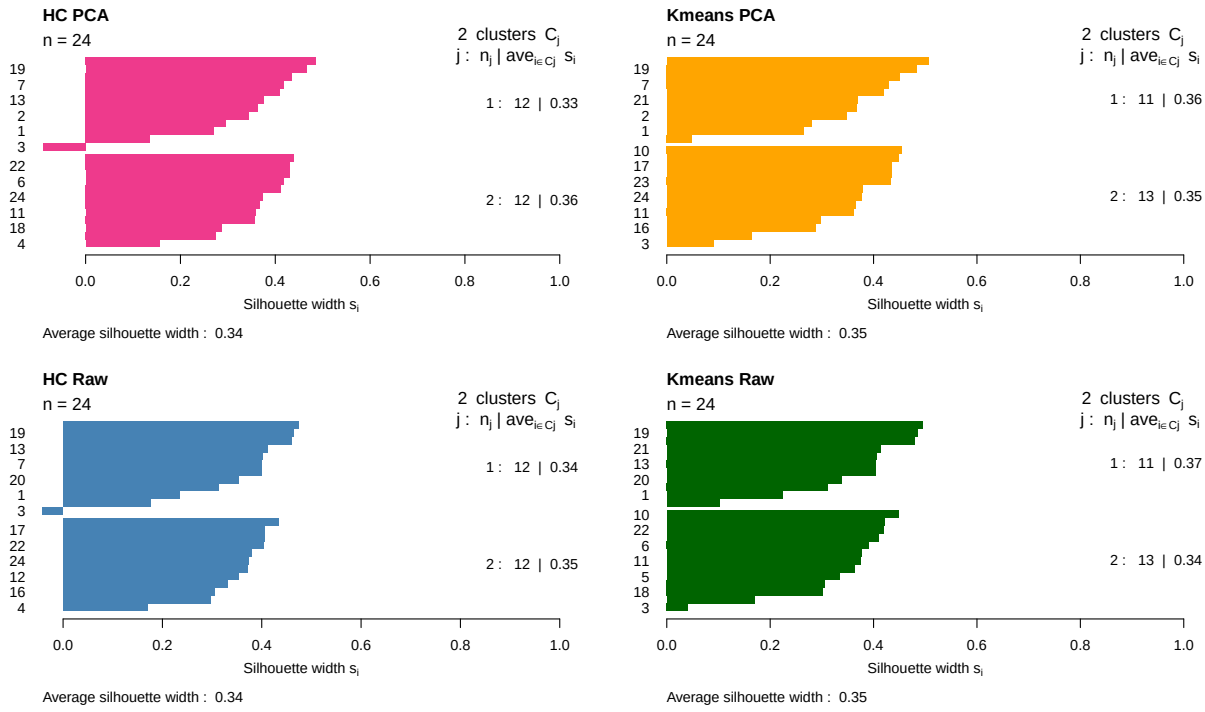
```
## Kmeans PCA ARI: -0.01055468
## HC PCA ARI: 0.001405429
## Kmeans Raw ARI: -0.01317217
## HC Raw ARI: -0.005287065
```

```
# Hierarchical and Kmeans Clustering normalized Wild Type
ClustersPCA(tmmWt,2, LabelsWT)
```



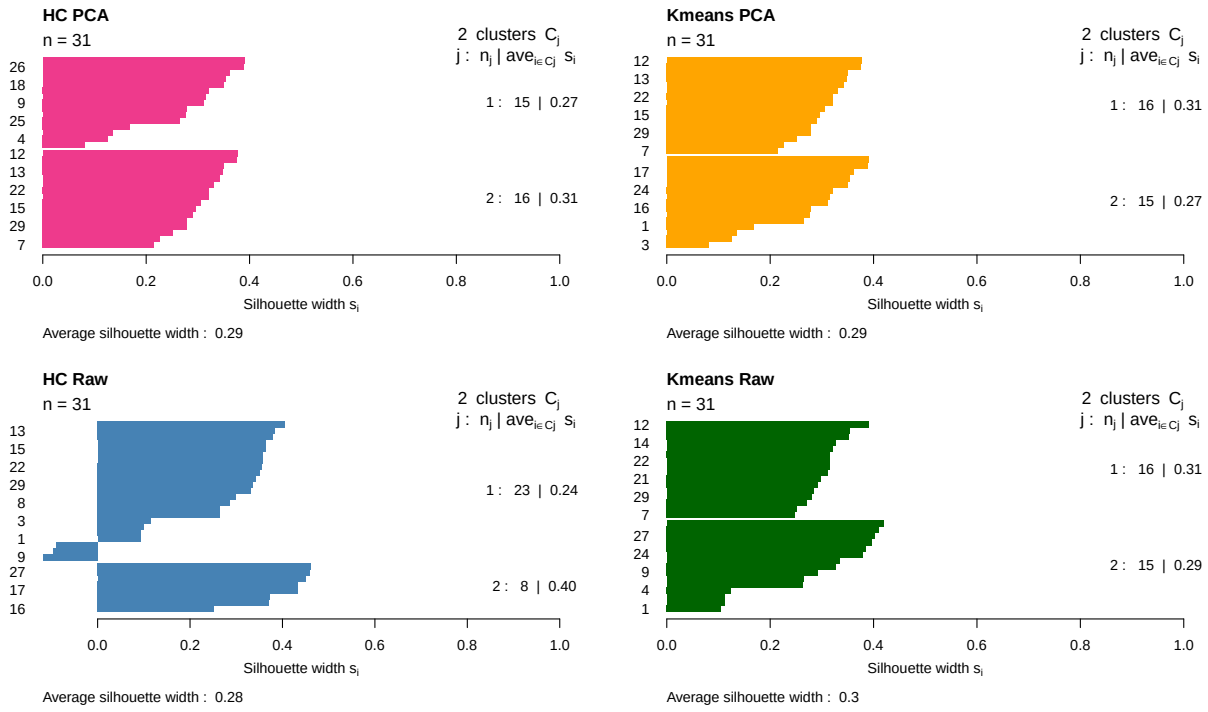

```
## Kmeans PCA ARI: 1
## HC PCA ARI: 1
## Kmeans Raw ARI: 1
## HC Raw ARI: 0.3567568
```

```
# Hierarchical and Kmeans Clustering normalized MED12
ClustersPCA(tmmMED12, 2, LabelsMED12)
```




```
## Kmeans PCA ARI: 0.8330704
## HC PCA ARI: 1
## Kmeans Raw ARI: 0.8330704
## HC Raw ARI: 1
```

```
# Hierarchical and Kmeans Clustering normalized MED24
ClustersPCA(tmmMED24,2, LabelsMED24)
```




```
## Kmeans PCA ARI: 1
## HC PCA ARI: 1
## Kmeans Raw ARI: 1
## HC Raw ARI: 0.2822492
```

Clustering Validation

Analysis of the pooled dataset (n = 76) revealed that global transcriptomic variance entirely masked temporal patterns; both Hierarchical Clustering (HC) and K-means failed to replicate timepoint categories (ARI approx. 0.00). However, stratifying the data by genetic background again improved biological signals. For the Wild Type (n = 21) and MED24 KO (n = 31) cohorts, complete-linkage HC on raw data was highly vulnerable to high-dimensional noise, yielding severe misclassifications (ARI = 0.357 and 0.282, respectively). However, pre-filtering variables via Principal Component Analysis (PCA) successfully neutralized this background noise to achieve a flawless reconstruction of ARI = 1.000, but an average silhouette width of >= 0.29. On the other hand, K-means demonstrated superior clustering power by maintaining an ARI = 1.000 across both raw and PC spaces compared to Hierarchical clustering .

Conversely, the MED12 KO cohort (n = 24) obtained a better performance on hierarchical clustering over k-mean partitioning, both raw and PC space HC achieved an ARI = 1.0, whereas K-means an ARI = 0.833.

These exhibits that while PCA dimensionality reduction is useful for stabilizing hierarchical frameworks against high-dimensional noise, the most accurate choice of clustering algorithm becomes highly variable across biological backgrounds.

Heatmaps

To visually analyze if expression patterns of the robust genes matched their temporal groups, we generated hierarchical heatmaps using “pheatmap” for each genetic background and “complete” as clustering method.

```r
library(ComplexHeatmap)
library(circlize)

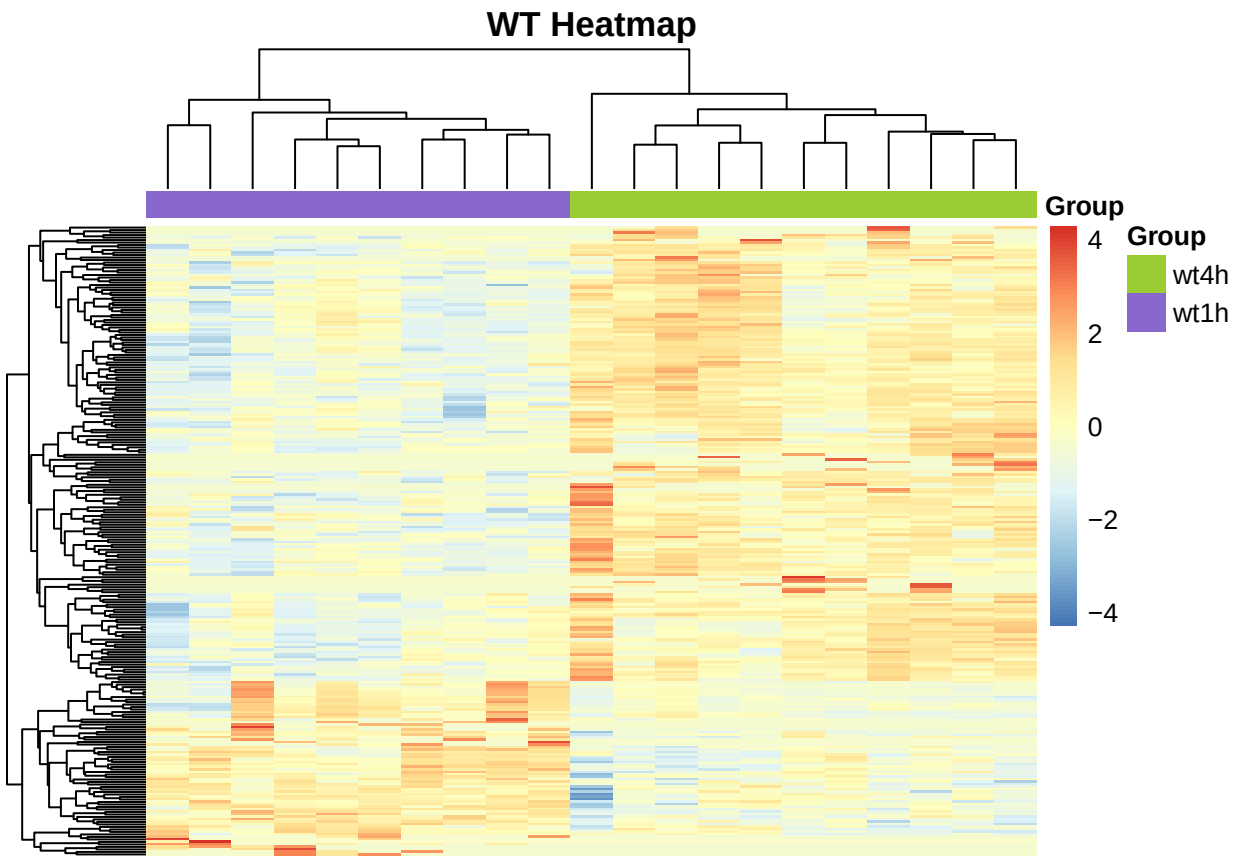
# WT
tmmWt <- tmm[ResultadosWT$common_genes, ExperimentalInfo$genotype == "wt"]

# MED12
tmmMED12 <- tmm[ResultadosM12$common_genes, ExperimentalInfo$genotype == "MED12"]

# MED24
tmmMED24 <- tmm[ResultadosM24$common_genes, ExperimentalInfo$genotype == "MED24"]

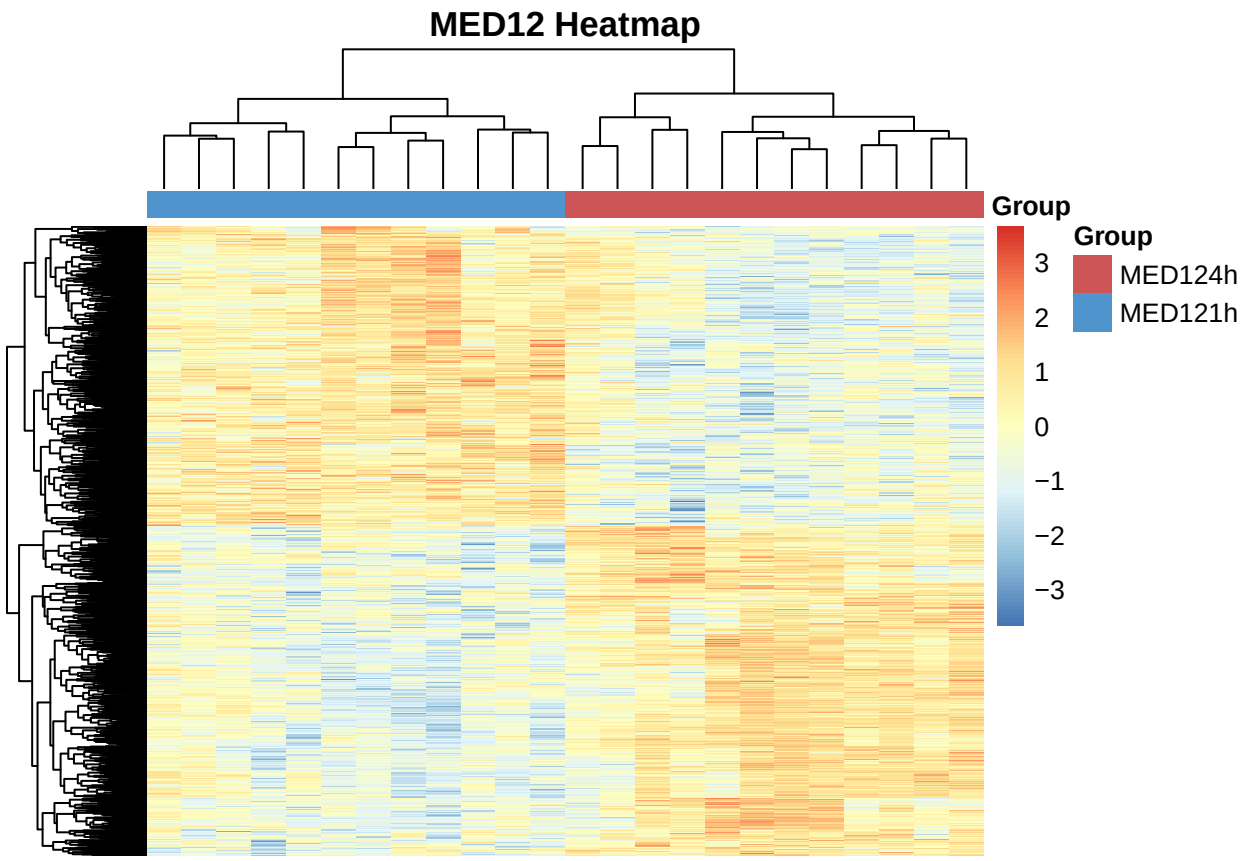
#colors configuration
ExperGroups <- sort(unique(ExperimentalInfo$Groups))
full_colors <- setNames(colores, ExperGroups)
par(mfrow = c(3,1))
#WT
anno_wt <- data.frame(Group = factor(ExperimentalInfo$Groups[ExperimentalInfo$genotype == "wt"]))
rownames(anno_wt) <- colnames(tmmWt)
groups_wt <- as.character(unique(anno_wt$Group))


pheatmap(log1p(tmmWt), main = "WT Heatmap", clustering_method = "complete",
         scale = "row", show_rownames = FALSE, show_colnames = FALSE,
         annotation_col = anno_wt,
         annotation_colors = list(Group = full_colors[groups_wt]),
         border_color = NA)
```



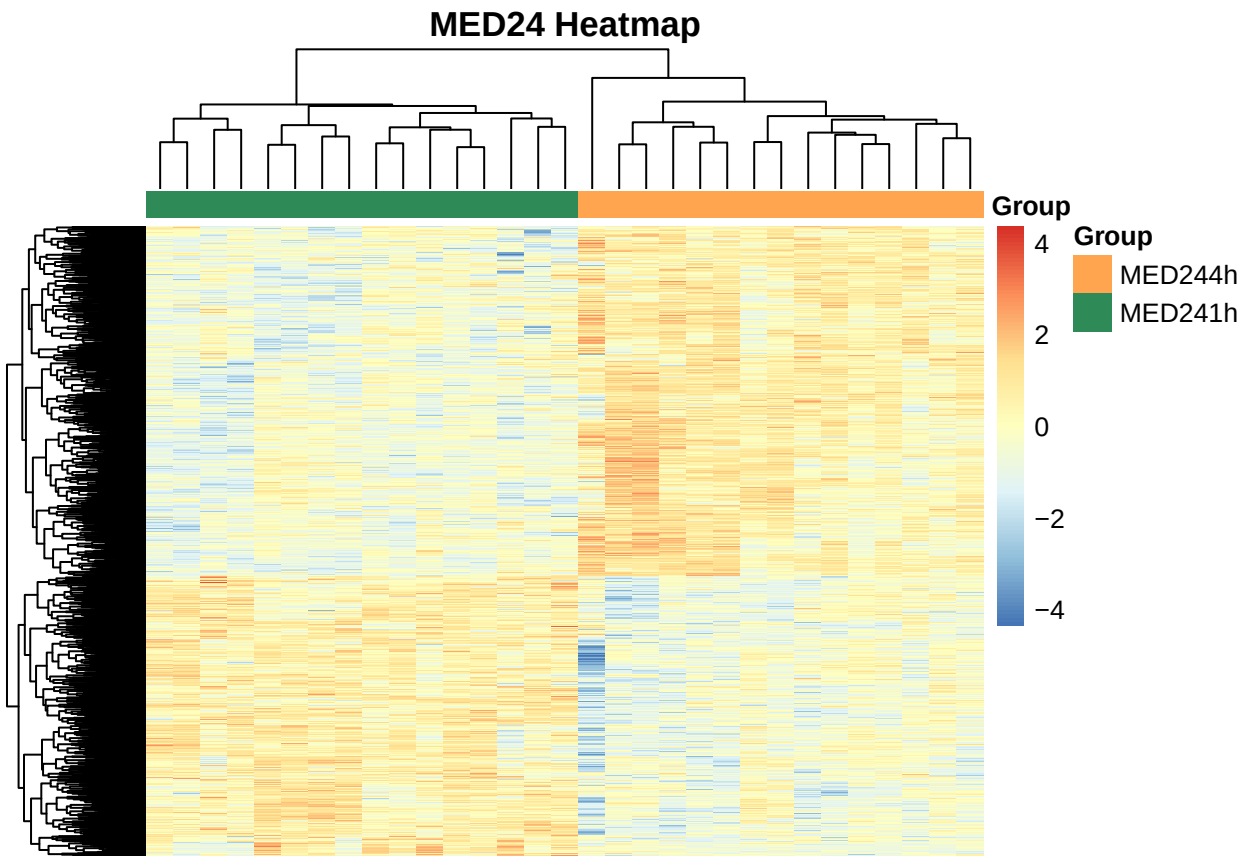

```
# MED12
anno_m12 <- data.frame(Group = factor(ExperimentalInfo$Groups[
                                ExperimentalInfo$genotype == "MED12"]))
rownames(anno_m12) <- colnames(tmmMED12)
groups_m12 <- as.character(unique(anno_m12$Group))

pheatmap(log1p(tmmMED12), main = "MED12 Heatmap", clustering_method = "complete",
         scale = "row", show_rownames = FALSE, show_colnames = FALSE,
         annotation_col = anno_m12,
         annotation_colors = list(Group = full_colors[groups_m12]),
         border_color = NA)
```




```
#MED24
anno_m24 <- data.frame(Group = factor(ExperimentalInfo$Groups[
                                    ExperimentalInfo$genotype == "MED24"]))
rownames(anno_m24) <- colnames(tmmMED24)
groups_m24 <- as.character(unique(anno_m24$Group))

pheatmap(log1p(tmmMED24), main = "MED24 Heatmap", clustering_method = "complete",
         scale = "row", show_rownames = FALSE, show_colnames = FALSE,
         annotation_col = anno_m24,
         annotation_colors = list(Group = full_colors[groups_m24]),
         border_color = NA, use_raster = TRUE)
```




```
par(mfrow = c(1,1))
```

The hierarchical heatmaps confirm that genetic background stratification uncovers a highly evident dichotomical structure between the 1h and 4h timepoints across all biological groups . For each matrix complete-linkage clustering using "complete" as clustering method achieves flawless separation of sample groups at k=2, aligning perfectly with the true biological labels.

Functional Over-Representation (ORA)

The primary objective of this section is to identify transcripts onto biological pathways, moving the resolution from individual genes to functional processes. Here we can obtain the change in functional landscape across timepoints within each of the three distinct genetic backgrounds.

We executed a three functional enrichment pipeline across the 3 genotypes to map transcriptomic differences during the 1h to 4h temporal points. First, using consensus robust genes we conducted a comparative over-representation analysis using KEGG, Reactome, and GO Biological Processes. Second, we applied Gene Set Enrichment Analysis on the mean logFC-ranked consensus gene lists from EdgeR and EDAseq and used GSEA ridgeline plots to visualize the distribution of gene expression changes ($logFC$) for statistically significant biological pathways. Third, from the unpooled logFC-ranked gene list (only DESeq2), we performed the following analysis: running enrichment score for I-kappaB kinase/NF-kappaB signaling (GO:0007249) and structural pathway mapping via "pathview" to visualize logFC expression dynamics directly onto the KEGG NF-kappa B signaling pathway (hsa04064).

```r
library(org.Hs.eg.db)
library(AnnotationDbi)
library(clusterProfiler)
library(ReactomePA)
library(pathview)
library(enrichplot)
library(ggplot2)
library(knitr)
library(patchwork)
library(ggplotify)
library(grid)
library(png)

universo <- rownames(AnnotatedWtDEA[[1]])

robust_M12 <- rownames(ResultadosM12$robust_logFC_matrix)
robust_M24 <- rownames(ResultadosM24$robust_logFC_matrix)
robust_WT  <- rownames(ResultadosWT$robust_logFC_matrix)

universo.entrez <- as.character(mapIds(org.Hs.eg.db, keys = universo,
                                       column = "ENTREZID", keytype = "ENSEMBL"))

M12entrez <- as.character(mapIds(org.Hs.eg.db, keys = robust_M12,
                                 column = "ENTREZID", keytype = "ENSEMBL"))
M24entrez <- as.character(mapIds(org.Hs.eg.db, keys = robust_M24,
                                 column = "ENTREZID", keytype = "ENSEMBL"))
WTentrez  <- as.character(mapIds(org.Hs.eg.db, keys = robust_WT,
                                 column = "ENTREZID", keytype = "ENSEMBL"))

clean_data <- function(x, ids) {
  valid <- !is.na(ids)
  x_filt <- x[valid]
  ids_filt <- ids[valid]

  dups <- duplicated(ids_filt)
  x_filt <- x_filt[!dups]
  ids_filt <- ids_filt[!dups]

  final_vector <- as.numeric(x_filt)
  names(final_vector) <- ids_filt

  return(final_vector)
}

gene_list_all <- list(M12 = M12entrez, M24 = M24entrez, WT = WTentrez)

cc_kegg <- compareCluster(geneClusters = gene_list_all, fun = "enrichKEGG",
                          organism = "hsa", universe = universo.entrez,
                          pAdjustMethod = "BH", pvalueCutoff = 0.05, qvalueCutoff = 0.2)

cc_reactome <- compareCluster(geneClusters = gene_list_all, fun = "enrichPathway",
                              organism = "human", universe = universo.entrez,
                              pAdjustMethod = "BH", pvalueCutoff = 0.05, qvalueCutoff = 0.2)
```

```r
run_cc_go <- function(ont) {
  compareCluster(geneClusters = gene_list_all, fun = "enrichGO",
                 OrgDb = org.Hs.eg.db, ont = ont, universe = universo.entrez,
                 pAdjustMethod = "BH", pvalueCutoff = 0.05, qvalueCutoff = 0.2, readable = TRUE)
}
```

```r
cc_go_bp <- run_cc_go("BP")
cc_go_bp_sim <- clusterProfiler::simplify(cc_go_bp, cutoff = 0.7, by = "p.adjust", select_fun = min)
```

```r
#gseGO
TopLogM12 <- clean_data(ResultadosM12$robust_logFC_matrix, M12entrez)
TopLogM24 <- clean_data(ResultadosM24$robust_logFC_matrix, M24entrez)
TopLogWt  <- clean_data(ResultadosWT$robust_logFC_matrix, WTentrez)

kkGOM12 = gseGO(geneList = TopLogM12, ont = "BP", OrgDb = 'org.Hs.eg.db',
                keyType = "ENTREZID", pAdjustMethod = "BH")
kkGOM24 = gseGO(geneList = TopLogM24, ont = "BP", OrgDb = 'org.Hs.eg.db',
                keyType = "ENTREZID", pAdjustMethod = "BH")
kkGOWT  = gseGO(geneList = TopLogWt,  ont = "BP", OrgDb = 'org.Hs.eg.db',
                keyType = "ENTREZID", pAdjustMethod = "BH")
```

```r
library(enrichplot)
library(clusterProfiler)

p1 <- dotplot(cc_go_bp_sim, showCategory = 5, font.size = 7) + ggtitle("Simplified GO BP")
p2 <- dotplot(cc_reactome, showCategory = 5, font.size = 7) + ggtitle("Reactome")
p3 <- dotplot(cc_kegg, showCategory = 5, font.size = 7) + ggtitle("KEGG")

p4 <- ridgeplot(kkGOM24, label_format = 50, showCategory = 10) + ggtitle("Med24 DESeq2")
p5 <- ridgeplot(kkGOM12, label_format = 50, showCategory = 10) + ggtitle("Med12 DESeq2")
# not used because no processes surpass the threshold
#p6 <- ridgeplot(kkGOWT, label_format = 50, showCategory = 10) + ggtitle("WT DESeq2") not used because
```

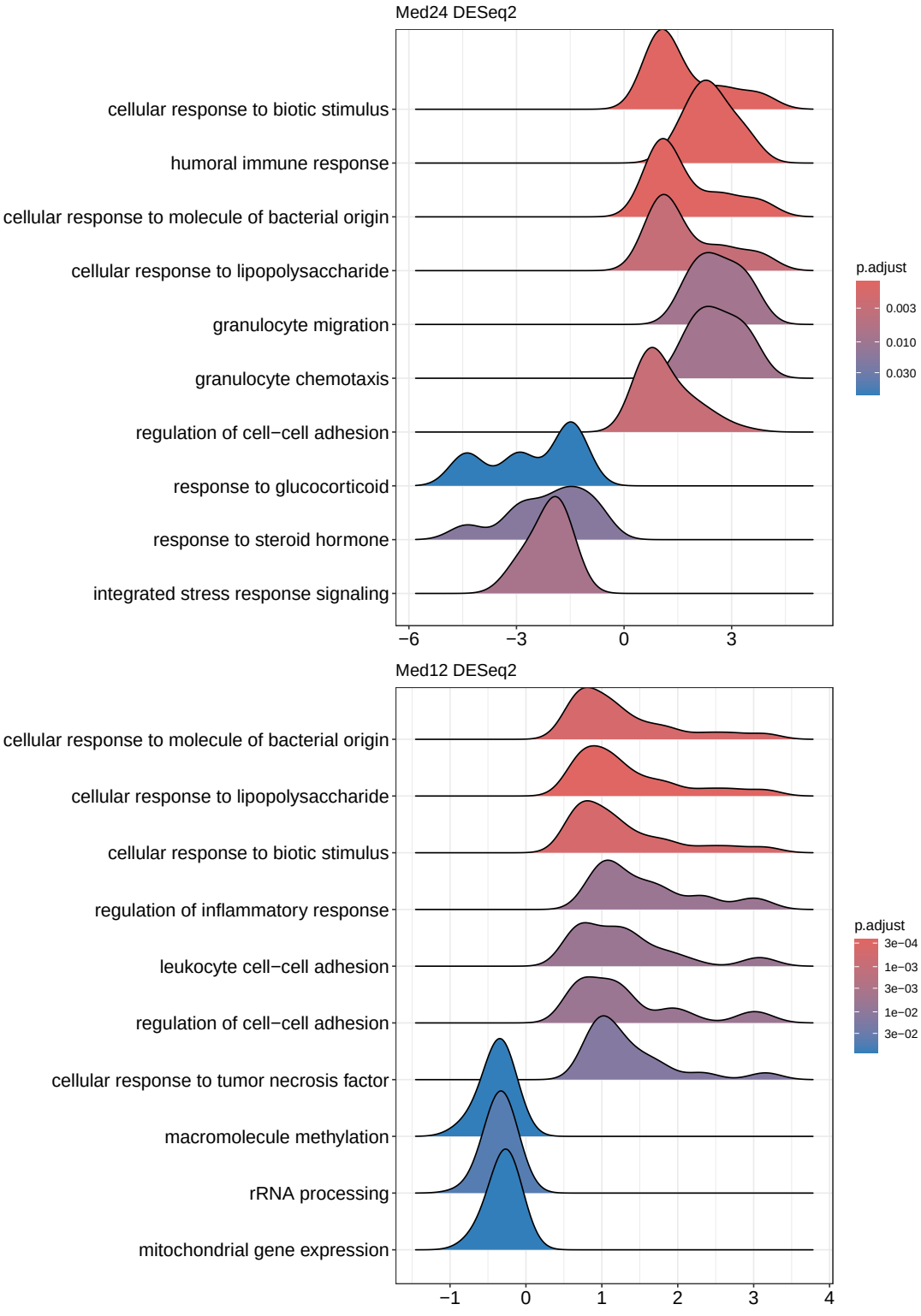
```r
p1/p2/p3
```


Simplified GO BP
extrinsic apoptotic signaling pathway
antigen processing and presentation of endogenous peptide antigen via MHC class I
regulation of innate immune response
regulation of response to biotic stimulus
small GTPase−mediated signal transduction
response to type II interferon
cytokine−mediated signaling pathway
response to tumor necrosis factor
cellular response to tumor necrosis factor
negative regulation of cell population proliferation
positive regulation of programmed cell death
regulation of inflammatory response
M12 (1398)
M24 (1525)
WT (216)
Cluster
p.adjust
1e−05
1e−04
1e−03
1e−02
GeneRatio
0.025
0.050
0.075
0.100
Reactome
Interferon gamma signaling
Interferon alpha/beta signaling
RHOB GTPase cycle
Death Receptor Signaling
RHOC GTPase cycle
Antigen Presentation: Folding, assembly and peptide loading of class I MHC
Toll Like Receptor 3 (TLR3) Cascade
MyD88−independent TLR4 cascade
TRIF (TICAM1)−mediated TLR4 signaling
TNFR1−induced proapoptotic signaling
Regulation of TNFR1 signaling
M12 (1003)
M24 (1073)
WT (151)
Cluster
p.adjust
1e−06
1e−04
1e−02
GeneRatio
0.02
0.03
0.04
0.05
KEGG
Epstein−Barr virus infection
Herpes simplex virus 1 infection
Kaposi sarcoma−associated herpesvirus infection
Human cytomegalovirus infection
NOD−like receptor signaling pathway
TNF signaling pathway
Pathways in cancer
IL−17 signaling pathway
M12 (719)
M24 (784)
WT (116)
Cluster
p.adjust
1e−06
1e−05
1e−04
1e−03
1e−02
GeneRatio
0.04
0.08
0.12
0.16

Over-representation dotplots

Changes from 1 hour to 4 hour across all genotypes is orchestrated by an activation of innate immune cascades, among them TNF and NOD-like receptor pathways . The over-representation dotplots generated via obtained from "ClusterProfiler" and "enrichplot", show that mutating specific Mediator subunits alters the downregulation of proinflamatory response. The MED24 KO displays a hyper-inflammatory profile, characterized by a decoupling between innate detection and adaptive immune bridging, showing an exclusively enriching for MyD88-independent TLR4 signaling profile, while not displaying nor MHC Class I antigen presentation or Interferon alpha/beta pathways.

```
p4/p5
```


Med24 DESeq2
cellular response to biotic stimulus
humoral immune response
cellular response to molecule of bacterial origin
cellular response to lipopolysaccharide
granulocyte migration
granulocyte chemotaxis
regulation of cell−cell adhesion
response to glucocorticoid
response to steroid hormone
integrated stress response signaling
−6
−3
0
3
p.adjust
0.003
0.010
0.030
Med12 DESeq2
cellular response to molecule of bacterial origin
cellular response to lipopolysaccharide
cellular response to biotic stimulus
regulation of inflammatory response
leukocyte cell−cell adhesion
regulation of cell−cell adhesion
cellular response to tumor necrosis factor
macromolecule methylation
rRNA processing
mitochondrial gene expression
−1
0
1
2
3
4
p.adjust
3e−04
1e−03
3e−03
1e−02
3e−02

GSEA ridgeline and Pathview

The GSEA ridgeline plots show that these immune clusters are actively upregulated over time, driven by an activation of responses to bacterial "like" molecules. Meanwhile MED12 shows leukocyte adhesion and maintain a stable baseline for housekeeping processes like rRNA processing. MED24 shifts toward granulocyte migration and active suppression of protective networks, among them: the integrated stress response. This highlights that MED12 volatility is tightly restricted to acute immune cascades, while MED24 is essential for sustaining cellular survival pathways under inflammatory stress.

```
library(ggplotify)
library(patchwork)
library(grid)
library(png)
library(pathview)
#  Deseq retrieval and vectorization
DeseqLogRankWT <- as.vector(AnnotatedWtDEA[[2]]$logFC)
DeseqLogRankM12 <- as.vector(AnnotatedM12DEA[[2]]$logFC)
DeseqLogRankM24 <- as.vector(AnnotatedM24DEA[[2]]$logFC)

DeseqWTentrez <- as.character(mapIds(org.Hs.eg.db, keys = rownames(AnnotatedWtDEA[[2]]),
                                     column = "ENTREZID", keytype = "ENSEMBL"))
DeseqM12entrez <- as.character(mapIds(org.Hs.eg.db, keys = rownames(AnnotatedM12DEA[[2]]),
                                      column = "ENTREZID", keytype = "ENSEMBL"))
DeseqM24entrez <- as.character(mapIds(org.Hs.eg.db, keys = rownames(AnnotatedM24DEA[[2]]),
                                      column = "ENTREZID", keytype = "ENSEMBL"))

DeseqLogRankWT <- clean_data(DeseqLogRankWT, DeseqWTentrez)
DeseqLogRankM12 <- clean_data(DeseqLogRankM12, DeseqM12entrez)
DeseqLogRankM24 <- clean_data(DeseqLogRankM24, DeseqM24entrez)


kkGOM12 <- gseGO(geneList = DeseqLogRankM12, ont = "BP", OrgDb = 'org.Hs.eg.db',
                 keyType = "ENTREZID", pAdjustMethod = "BH")
kkGOM24 <- gseGO(geneList = DeseqLogRankM24, ont = "BP", OrgDb = 'org.Hs.eg.db',
                 keyType = "ENTREZID", pAdjustMethod = "BH")
kkGOWT  <- gseGO(geneList = DeseqLogRankWT,  ont = "BP", OrgDb = 'org.Hs.eg.db',
                 keyType = "ENTREZID", pAdjustMethod = "BH")

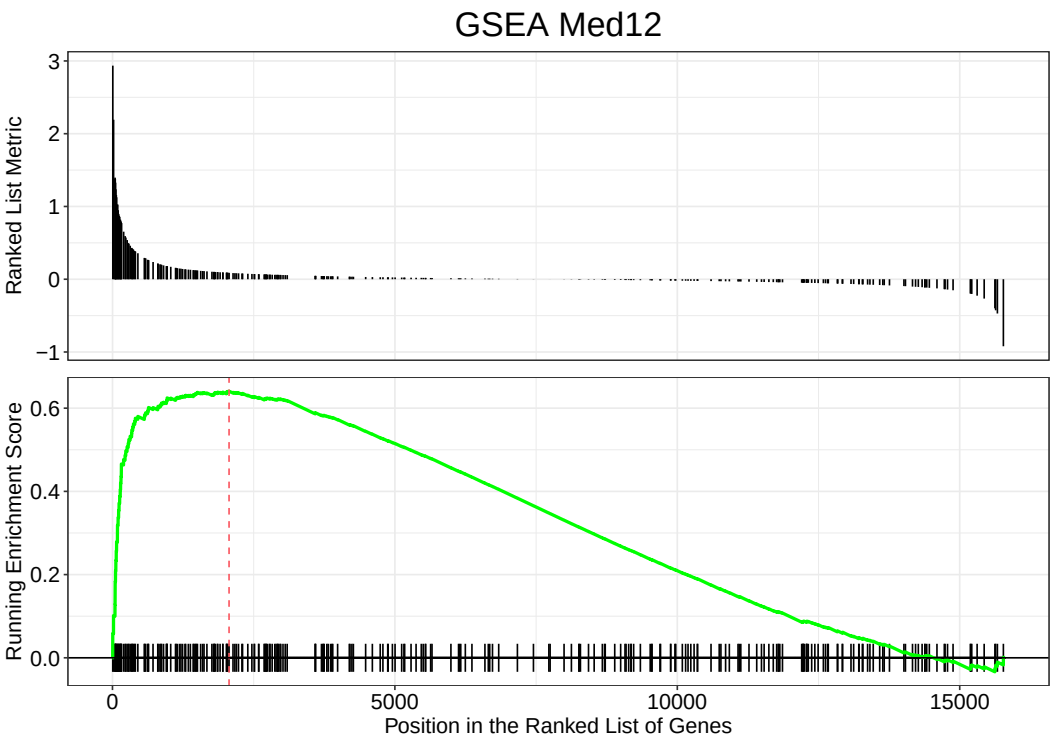
P1 <- as.ggplot(gseaplot(kkGOM12, geneSetID = "GO:0007249", title = "GSEA Med12"))
P2 <- as.ggplot(gseaplot(kkGOM24, geneSetID = "GO:0007249", title = "GSEA Med24"))
P3 <- as.ggplot(gseaplot(kkGOWT, geneSetID = "GO:0007249", title = "GSEA WT"))

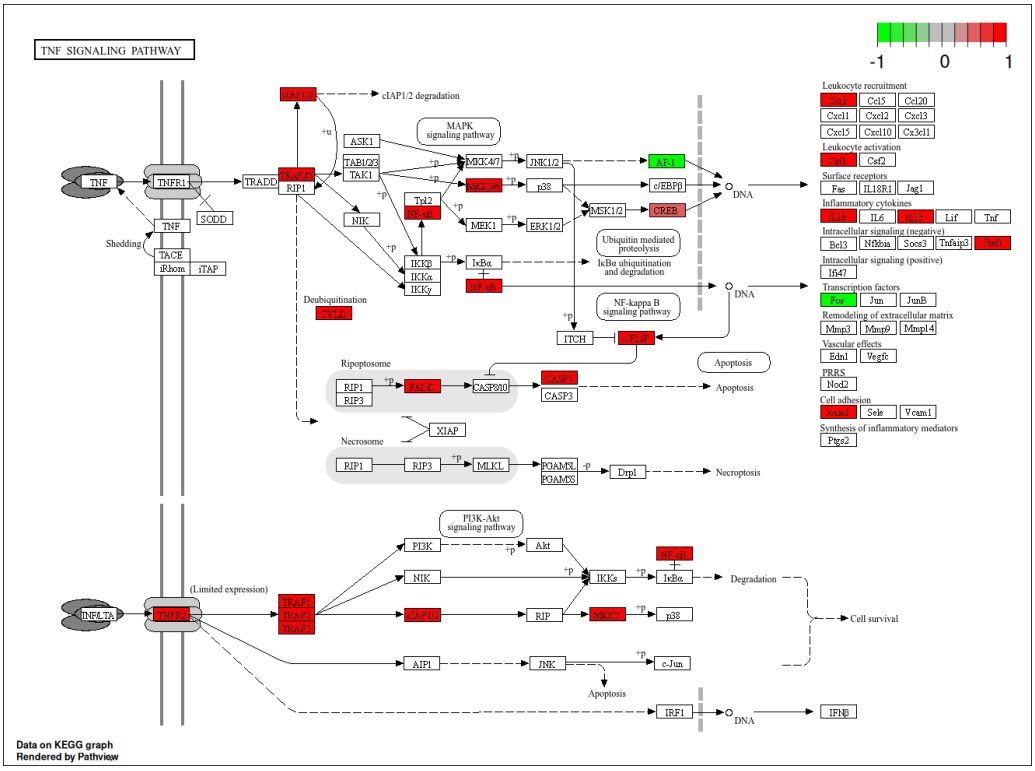

# Pathview
pathview(gene.data = DeseqLogRankM12, pathway.id = "hsa04064",
         species = "hsa", out.suffix = "M12.pathview")
pathview(gene.data = DeseqLogRankM24, pathway.id = "hsa04064",
         species = "hsa", out.suffix = "M24.pathview")
pathview(gene.data = DeseqLogRankWT,  pathway.id = "hsa04064",
         species = "hsa", out.suffix = "WT.pathview")


# Reading PNGs
img1 <- rasterGrob(readPNG("hsa04064.M12.pathview.png"), interpolate = TRUE)
```

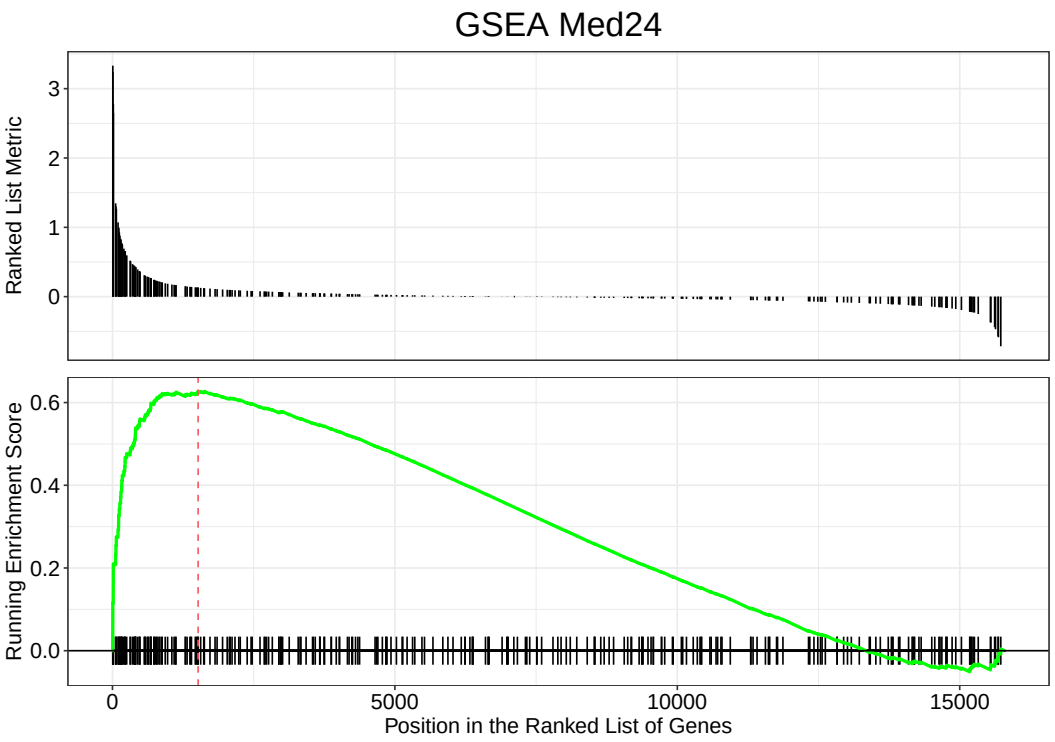
```r
img2 <- rasterGrob(readPNG("hsa04064.M24.pathview.png"), interpolate = TRUE)
img3 <- rasterGrob(readPNG("hsa04064.WT.pathview.png"), interpolate = TRUE)
```

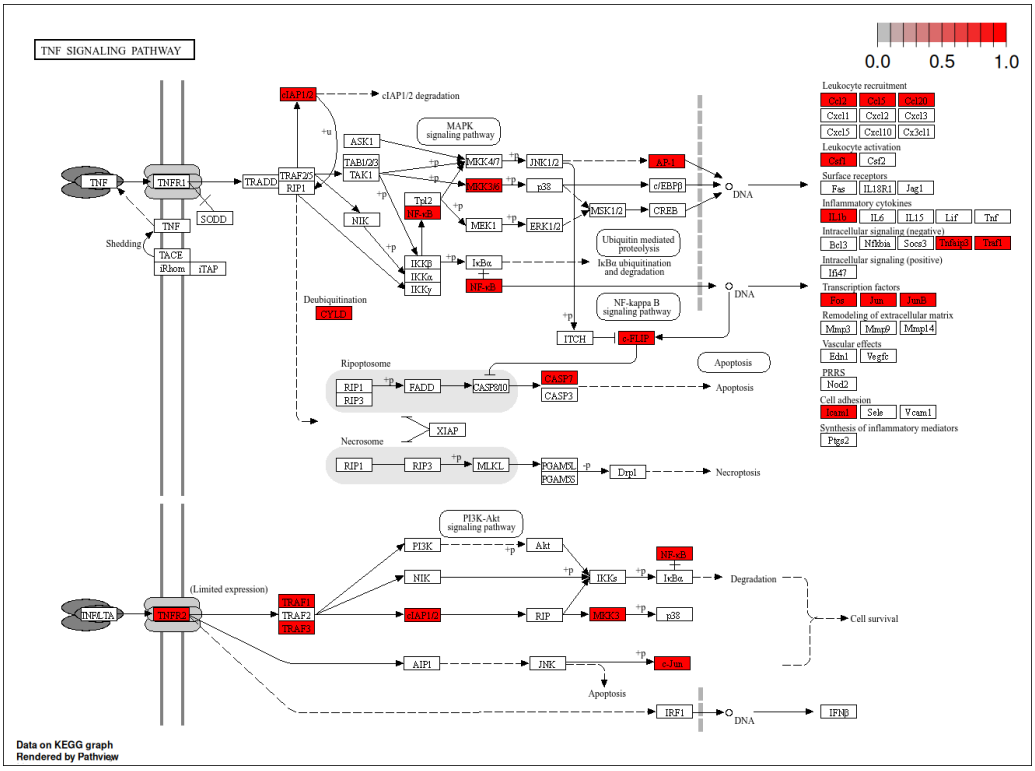
```r
P1 / wrap_elements(img1)
```



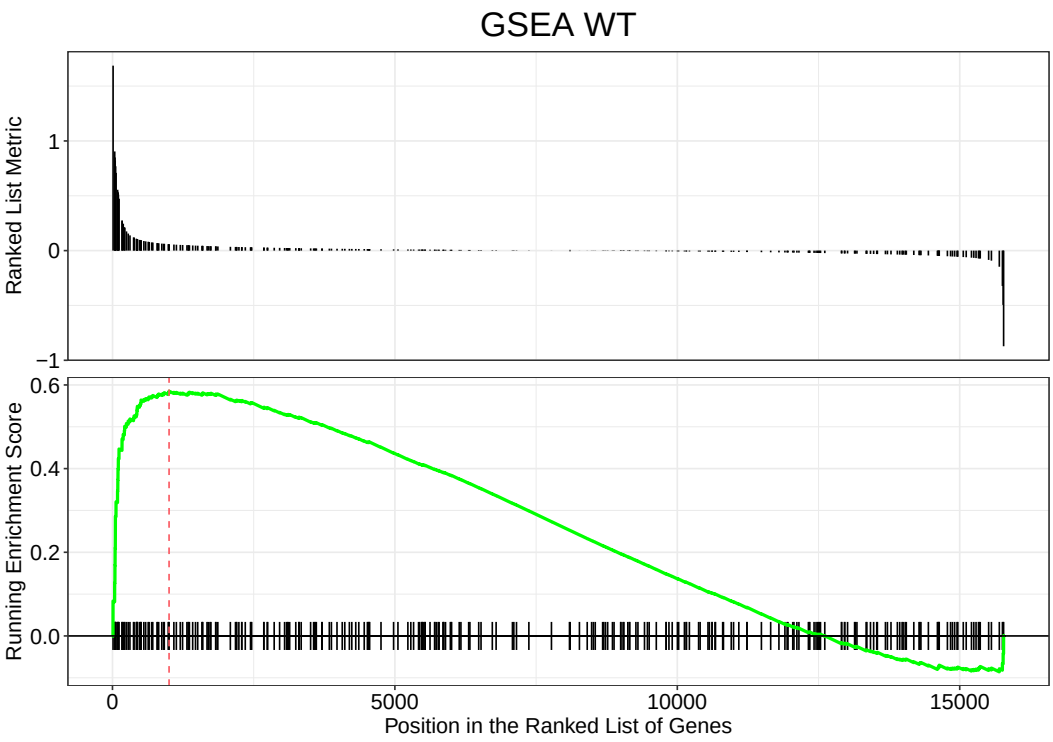


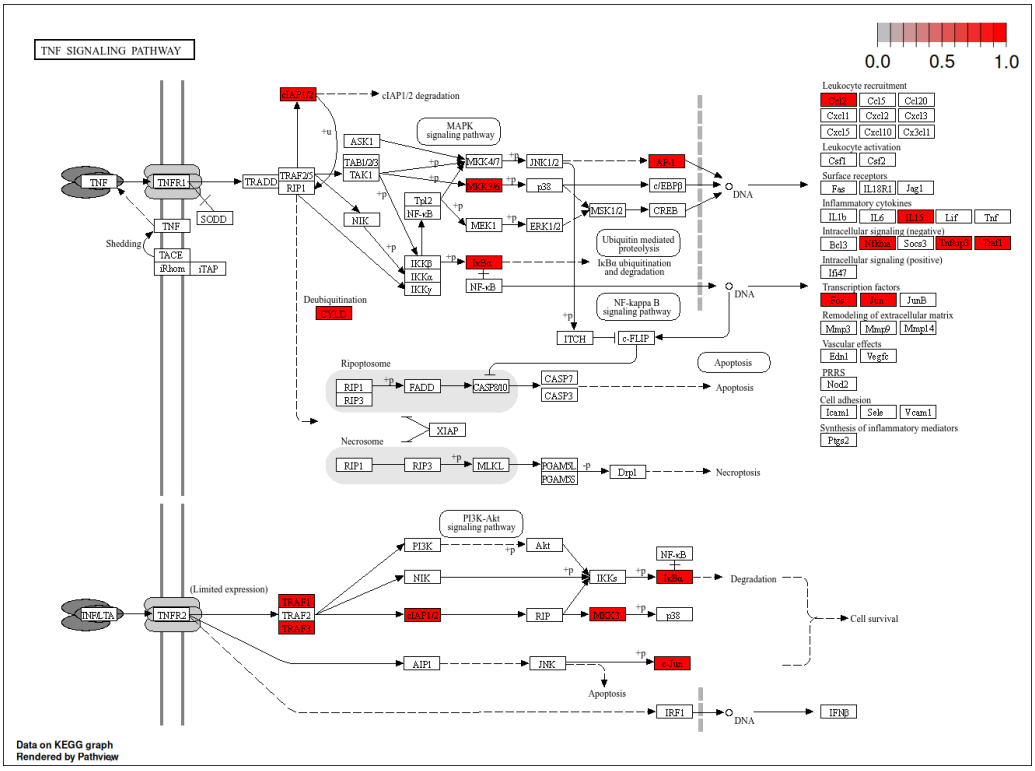

```
P2 / wrap_elements(img2)
```






```
P3/  wrap_elements(img3)
```





Evaluation of the NF-kB pathway The running enrichment scores show a significant enrichment wave across all three genotypes, peaking at the top of upregulated genes. To map these changes, Pathview was leveraged to project these differential expression values on the KEGG NF-kB signaling pathway diagram,

rendering upregulated nodes in red and downregulated nodes in green. In the running enrichment score the MED24 KO mutant reaches a remarkably high and compressed enrichment score peak just bellow 0.8, with deregulated pathway nodes including TRAF, cIAP, and the IKK complex, which light up heavily in red.

Implementation and comparison of different Bioconductor packages

We aimed to employ alternative Differential Expression Analysis packages to evaluate their respective results and identify a cross-validated set of genes among them.

We achieved this by implementing both parametric and non-parametric approaches. We used edgeR as a benchmark reference, alongside frameworks like DESeq2 and NOISeq. While both edgeR and DESeq2 are parametric tools that rely on a theoretical Negative Binomial curve we also implemented a shrinkage with DESeq2 to obtain further refined results from this package. We also implemented NOISeq (specifically the NOISeqBIO framework) to obtain a non-parametric reference.

EdgeR

1. Initial data normalization via TMM Before statistical modeling, edgeR normalizes the raw count matrix using the Trimmed Mean of M-values (TMM) method. This algorithm calculates sample-specific scaling factors by computing a weighted mean of log-expression ratios after stripping away the most extremely expressed genes and those with extreme log-fold changes. By ensuring that a few highly abundant master transcripts do not mathematically distort the relative sequencing depth available for the rest of the genome, TMM establishes a highly balanced starting point scale across all libraries.
2. Modeling Biological Variance via the Negative Binomial Distribution Because RNA-Seq datasets consist of discrete, non-negative integer read counts, continuous normal distributions are statistically inappropriate. While simple technical sequencing replication variance follows a Poisson distribution, true biological replicates introduce an additional layer of variation known as overdispersion in which the variance exceeds the mean expression levels, this makes the Negative Binomial (NB) distribution the right distribution to picture the biological variability.
3. Empirical Bayes Dispersion Moderation: The Squeeze Mechanism Estimating an independent dispersion value for over 15,000 individual genes using only the sample sizes is prone to error. EdgeR overcomes this limitation, by executing an Empirical Bayes (EB) moderation step, conceptually referred to as the "Squeeze" of dispersion across the entire transcriptome.

The mechanism follows a three-step algorithmic sequence:

The software calculates simultaneously a starting trend across all 15,000 genes named Global Trend Estimation. This captures that low-count genes naturally display higher relative variance. After it evaluates the raw, individual variance profile specifically exhibited by each individual gene across its respective replicates this process ending up with a Gene-wise Dispersion Estimation. Finally, edgeR applies a weighted conditional likelihood approach to mathematically transform and compress (squeeze) the unstable individual Gene-wise dispersions toward the global trend.This mechanism balances the dataset. Outlier genes exhibiting extreme, chaotic variance due to technical noise are scaled down to match the consensus trend. However, genes showing unusual low variance are scaled up. The end product in edgeR stabilizes variance estimations and establishes a foundation for the final Generalized Linear Model (GLM) Likelihood Ratio Tests (glmLRT).

DESeq2

DESeq2 utilizes an alternative normalization algorithm and a specialized shrinkage pipeline focused on effect-size calibration.

1. Initial Data Normalization via Median-of-Ratios DESeq2 normalizes raw count data using the Median-of-Ratios method, this framework computes scaling facators which are sample-specific (size factors), this is achieved by first calculating the geometric mean of each gene across all samples, which is used as a reference sample. Then the ratio of each sample's counts to this reference is obtained and extracts the median of these ratios. Because it relies on the median, this normalization is very robust against individual outliers and composition biases.
2. Modeling Biological Variance via the Negative Binomial Distribution Mirroring the edgeR framework, DESeq2 also fits the the discrete read counts to a Negative Binomial (NB) distribution
3. Log-Fold Change Moderation: The Shrinkage Mechanism The challenge of obtaining stable log-fold change for thousands of genes, particularly those with low read counts, is overcomed by executing an Empirical Bayes moderation step focused on the "Shrinkage" of Log-Fold Changes (utilizing the adaptive shrinkage method (ashr), which is an algorithm that looks at the unique standard error of each individual gene and tailors the shrinkage dynamically).

The mechanism follows:

Deseq2 as EdgeR also calculates a global baseline trend for 15,000 genes simultaneously. This outputs the distribution models the expected distribution of log-fold changes under the conservative assumption that the majority of genes do not change significantly.The Deseq2 library evaluates the log-fold change and standard error exhibited by each individual gene across its respective replicates and then a Shrinkage Transformation can then be optionally applied. This shrinkage mechanism balances the dataset based on the information available for each gene Low-expression or highly variable genes that show artificially extreme, chaotic fold changes due to random technical noise are shrunk. On the other hand, highly expressed genes with replicate stability are left almost unchanged, this drastically reduces false-positive rates and corrects for effect sizes, providing a reliable starting point for downstream pathway enrichment analysis.

NOiseq

NOISeq is a RNA-seq differential expression analysis non-parametric method that empirically models the noise distribution directly from the data.

The empirical pipeline is executed through three core phases:

1. Initial Data Normalization via TMM The raw counts are explicitly normalized using the Trimmed Mean of M-values (TMM) method within the noiseqbio function. This step ensures that baseline sequencing depth variations and compositional library biases are corrected, making the following empirical calculations directly comparable.
2. Empirical Noise Distribution Modeling The core philosophy of NOISeq defines background noise without relying on parametric distributions. For every gene, the algorithm tracks two primary metrics between conditions: M : The Log-Fold Change D: The Absolute Difference. The absolute change which is the raw number of reads gained or lost.

Noiseqbio recognices random variation by computing baseline M and D profiles and making all possible pairwise comparisons between biological replicates within the exact same experimental group. Because no true exogenous biological stimulus changes between replicates of the same group are expected, these variations are considered noise. By pooling these values across all features, the software constructs an empirical "noise distribution" unique to our dataset.

3. Probability-Based Hypothesis NOISeq compares the true M and D values obtained between our target conditions against the previously constructed empirical noise distribution. The software outputs a Posterior Probability of Differential Expression (q). This statistic measures the probability that a gene's expression change is driven by true experimental conditions rather than chance. A gene is flagged as a high-confidence if its M and D metrics are out of the boundaries of the noise matrix (utilizing a strict threshold of $q >= 0.95$).

Biological interpretation

The NF- B signaling pathway is a pillar of the cellular response to inflammatory stimuli, requiring a very rigorous control to prevent deleterious inflammation while also allowing its essential protective acute response. Among the components of this pathway, Feldman et al. (2019), have identified the Mediator complex, specifically the MED12 and MED24 subunits, as major negative regulators. These findings introduce an alternative model where the Mediator complex functions as a "transcriptional brake," regulating the duration and magnitude of pro-inflammatory gene expression. By comparing transcriptomic profiles of the M12, M24, and WT clusters, our results capture the transcriptional transition driven by Mediator subunit deficiency in inflammatory induced conditios. The proinflammatory landscape proposed by Feldman was coherent with our Over-Representation Analysis (ORA) results, including the presence of the canonical NF- B signaling pathway, leukocyte cell adhesion, and the regulation of inflammatory responses. Additionally, our results also provide information on the different transcription profiles between the MED12 KO, which exhibits stronger p-adjusted values and larger GeneRatios across specific inflammatory pathways like cellular response to lipopolysaccharide, while the GSEA ridge plots show that MED24 features a broader distribution of upregulated immune-related terms (humoral immune response, defense response to bacterium). This further aligns with Feldman's observation of the diferent contributions of both subunits. Our results also suggest that Mediator complex loss also enriches for TLR cascades, TNF receptor superfamily signaling, and cytokine-cytokine receptor interactions, and also for pro-apoptotic signatures, suggesting that the loss of these subunits may onset cells to programmed cell death. The GSEA Enrichment Plots for the NF-kappa B signaling pathway (KEGG: hsa04064)illustrate the transition from controlled physiological activation in Wild Type to hyper-induction. WT showed robust baseline activation curve peaking at arround 0.50, meanwhile MED12 and MED24 deficiency enrichment scores are peak around 0.70 and 0.75 respectively, both with a distinct leftward leading-edge shift. This is further evidenced in the Ranked List Metric y-axis, which surges from a maximum of approximately 1.5 in WT to more than 20.0 in both knockout cell lines. The convergence of these results show consistency with the model in which, when MED12 or MED24 are removed, this negative regulation is shattered, permitting the amplified and sustained expression of mediators that orchestrate chronic inflammation.